

http://algs4.cs.princeton.edu

4.2 DIRECTED GRAPHS

- *introduction*
- *digraph API*
- *digraph search*
- *topological sort*
- *strong components*

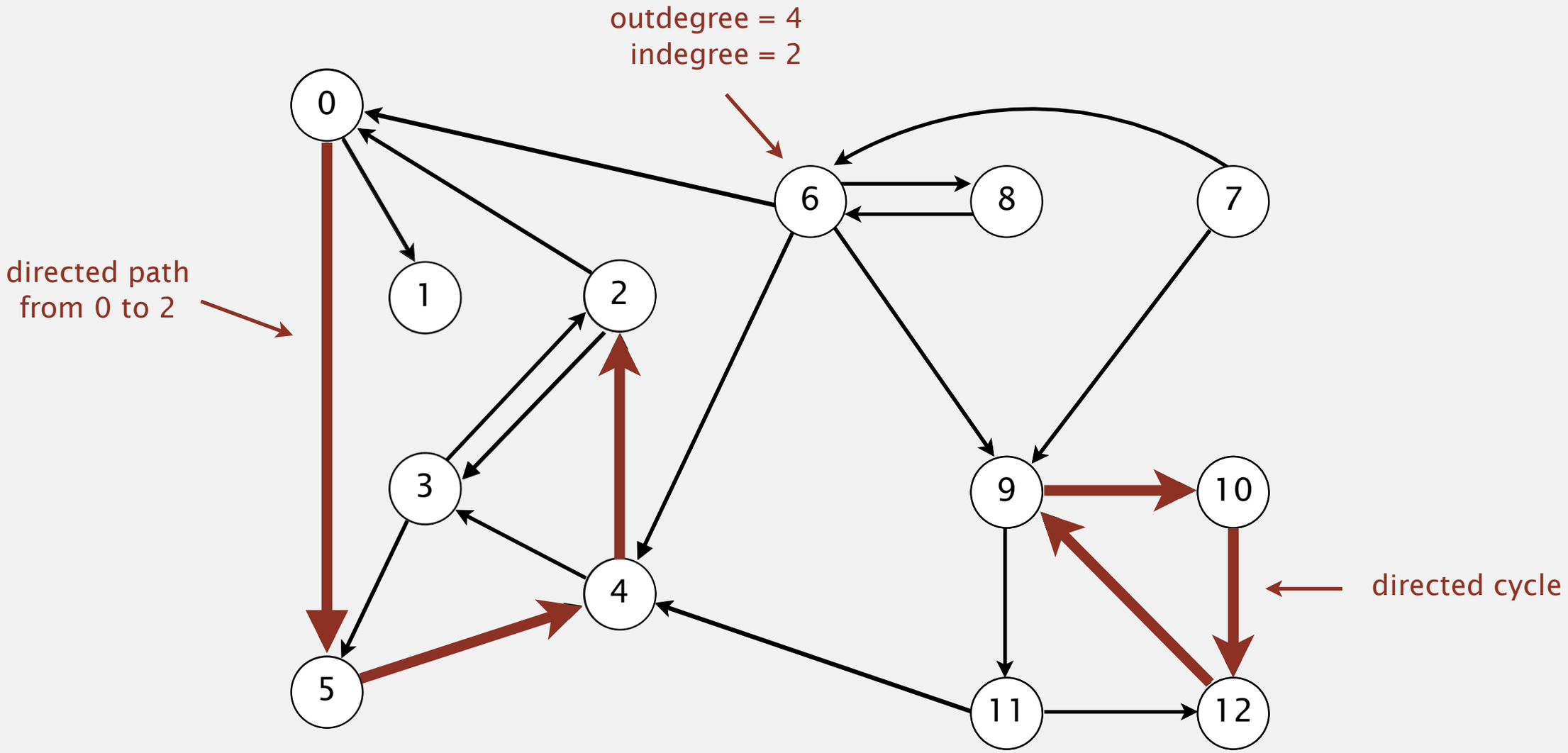


4.2 Directed Graphs

- *introduction*
- *digraph API*
- *digraph search*
- *topological sort*
- *strong components*

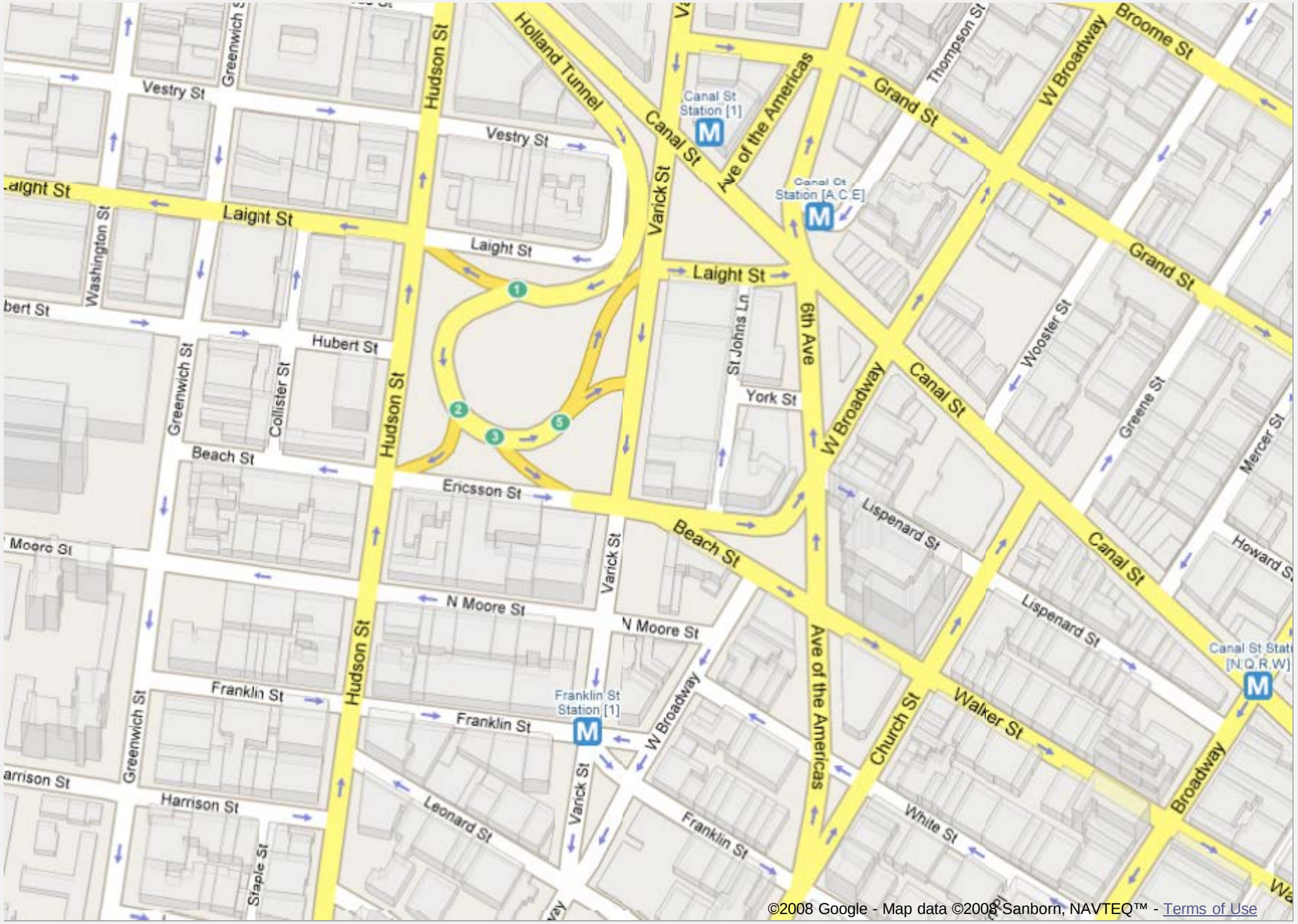
Directed graphs

Digraph. Bashkësi e kulmeve të lidhura në çift me anë të directed edges.



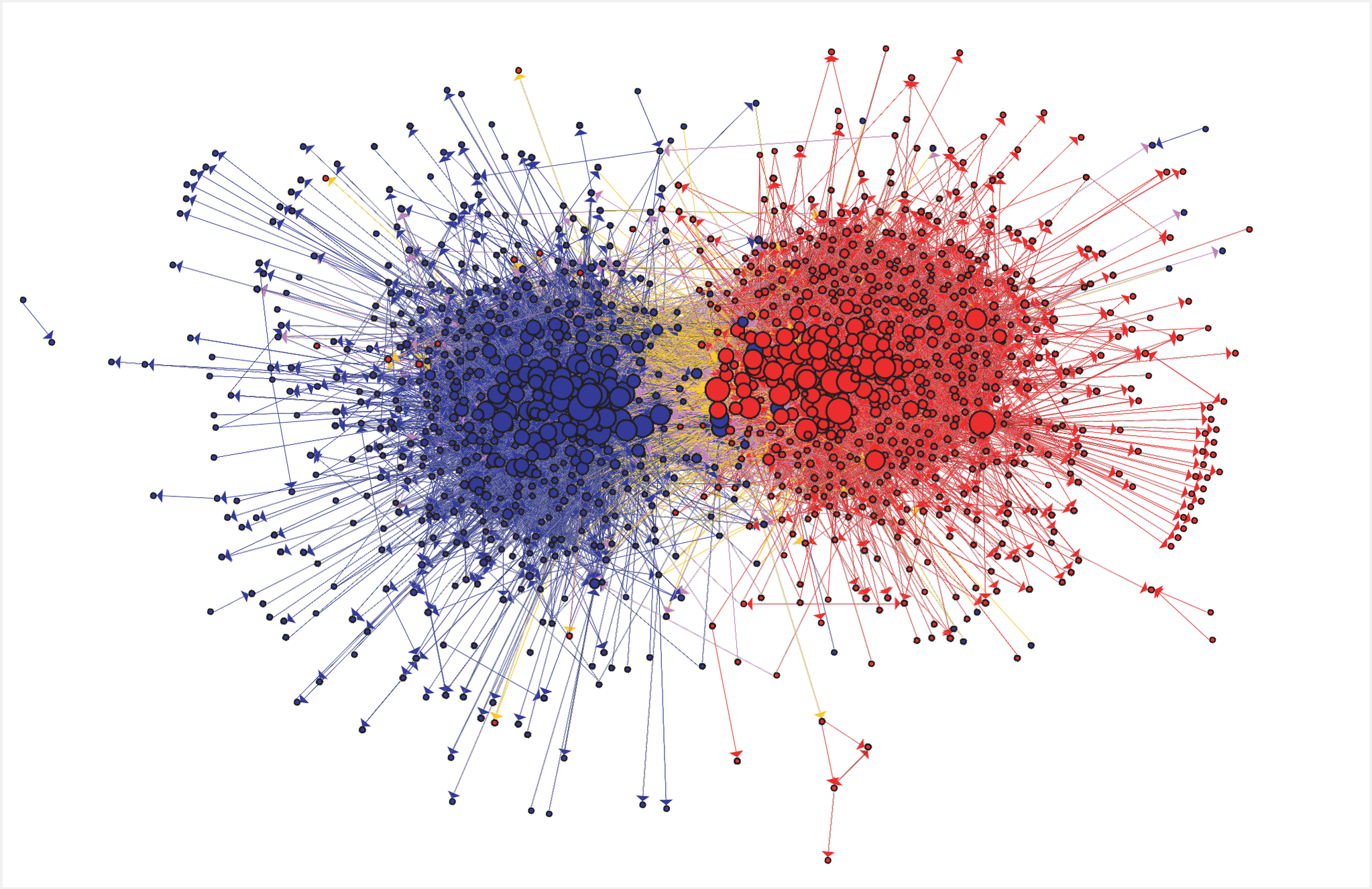
Rjeti i rrugëve

Kulmi = udhëkryq; segment = rruga njëkahore.



Grafi i blogosferës politike

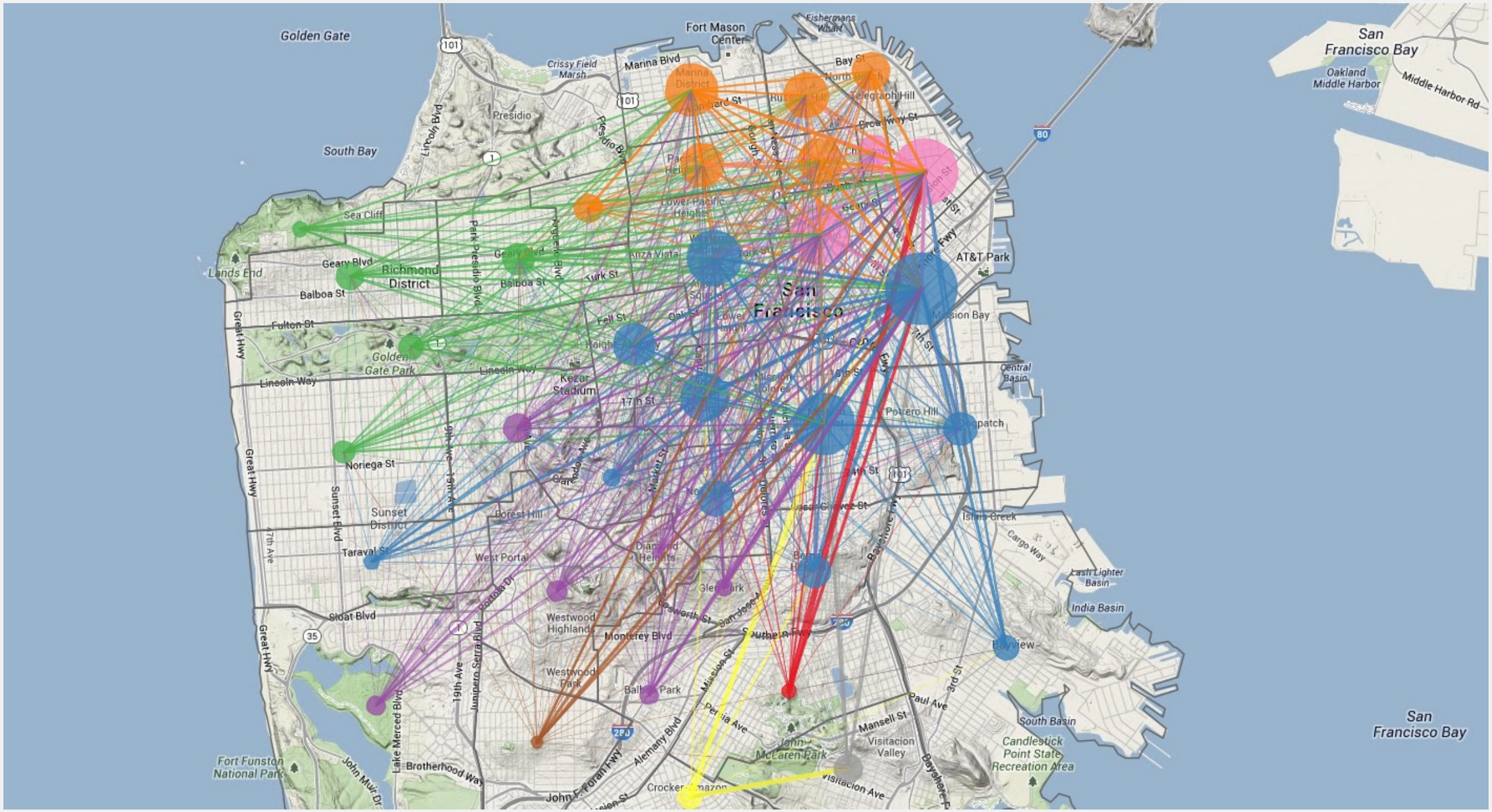
Kulmi= blog politik; segment = link.



The Political Blogosphere and the 2004 U.S. Election: Divided They Blog, Adamic and Glance, 2005

Uber taxi grafi

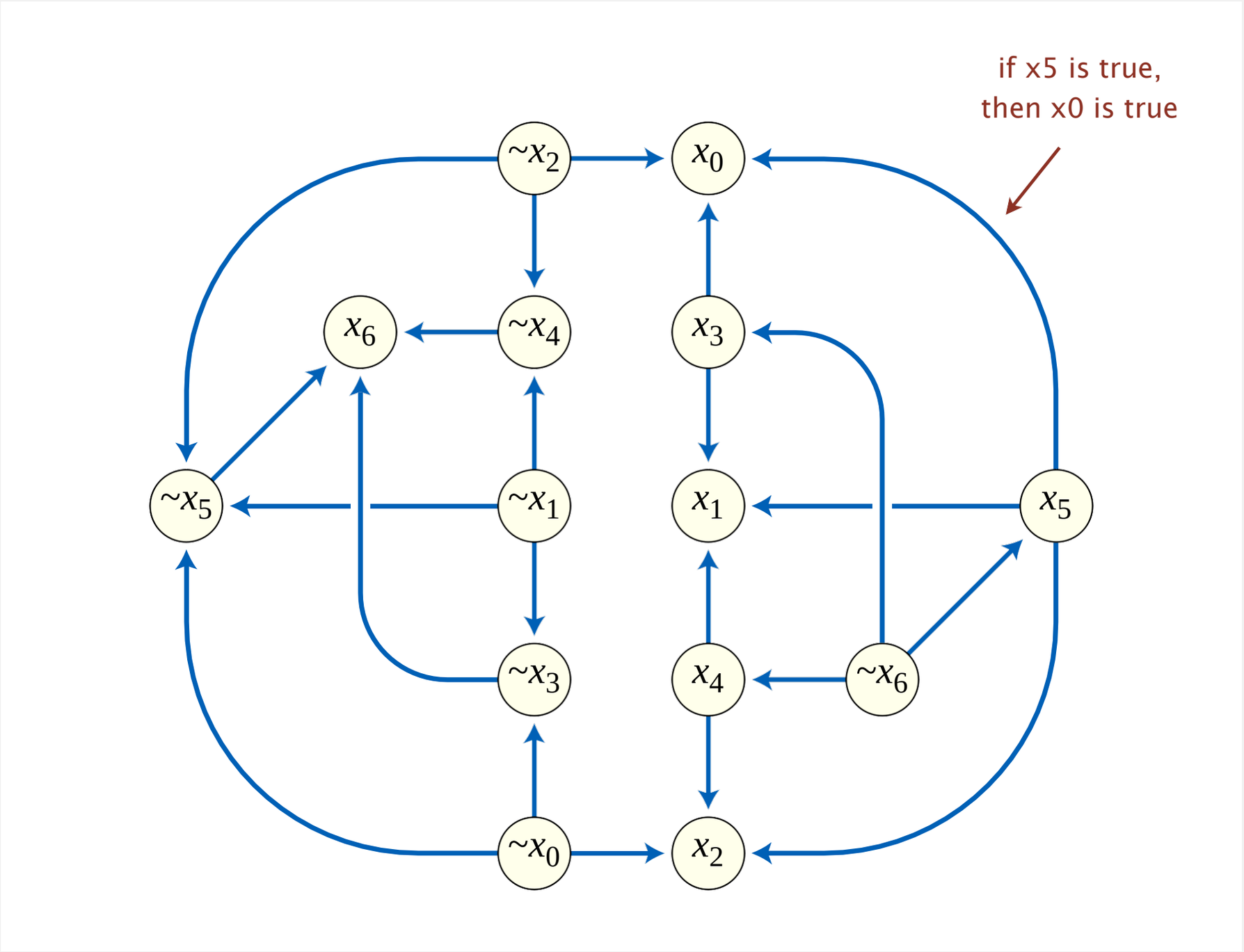
Kulmi = taxi marrja; edge = taxi vozitja.



http://blog.uber.com/2012/01/09/uberdata-san-franciscomics/

Grafi për implikim

Kulmi= variabla; segment = implikimi logjik.



Digraph applications

digraph	vertex	directed edge
transportation	street intersection	one-way street
web	web page	hyperlink
food web	species	predator-prey relationship
WordNet	synset	hypernym
scheduling	task	precedence constraint
financial	bank	transaction
cell phone	person	placed call
infectious disease	person	infection
game	board position	legal move
citation	journal article	citation
object graph	object	pointer
inheritance hierarchy	class	inherits from
control flow	code block	jump

Some digraph problems

problem	description
s→t path	*Is there a path from s to t ?*
shortest s→t path	*What is the shortest path from s to t ?*
directed cycle	*Is there a directed cycle in the graph ?*
topological sort	*Can the digraph be drawn so that all edges point upwards?*
strong connectivity	*Is there a directed path between all pairs of vertices ?*
transitive closure	*For which vertices v and w is there a directed path from v to w ?*
PageRank	*What is the importance of a web page ?*



4.2 Directed Graphs

- introduction
- digraph API
- digraph search
- topological sort
- strong components

Digraf API

Pothuajse identik me `Graf` API.

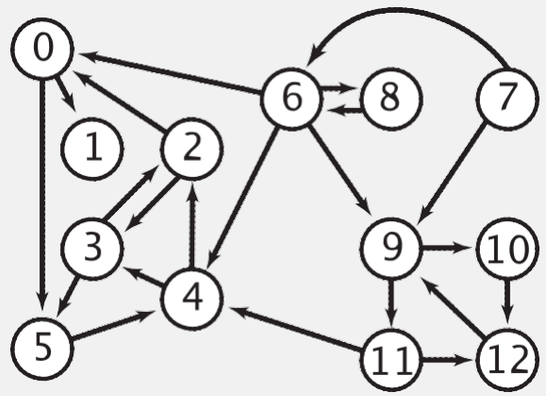
public class Digraph		
	`Digraph(int V)`	*create an empty digraph with V vertices*
	`Digraph(In in)`	*create a digraph from input stream*
`void`	`addEdge(int v, int w)`	*add a directed edge v→w*
`Iterable<Integer>`	`adj(int v)`	*vertices pointing from v*
`int`	`V()`	*number of vertices*
`int`	`E()`	*number of edges*
`Digraph`	`reverse()`	*reverse of this digraph*
`String`	`toString()`	*string representation*

Digraph API

tinyDG.txt

```
V
13
22
 4  2
 2  3
 3  2
 6  0
 0  1
 2  0
11 12
12  9
 9 10
 9 11
 7  9
10 12
11  4
 4  3
 3  5
 6  8
 8  6
 ⋮
```

E




```
% java Digraph tinyDG.txt
0->5

0->1
2->0
2->3
3->5
3->2
4->3
4->2
5->4
 ⋮
11->4
11->12
12->9
```

```
In in = new In(args[0]);
Digraph G = new Digraph(in);

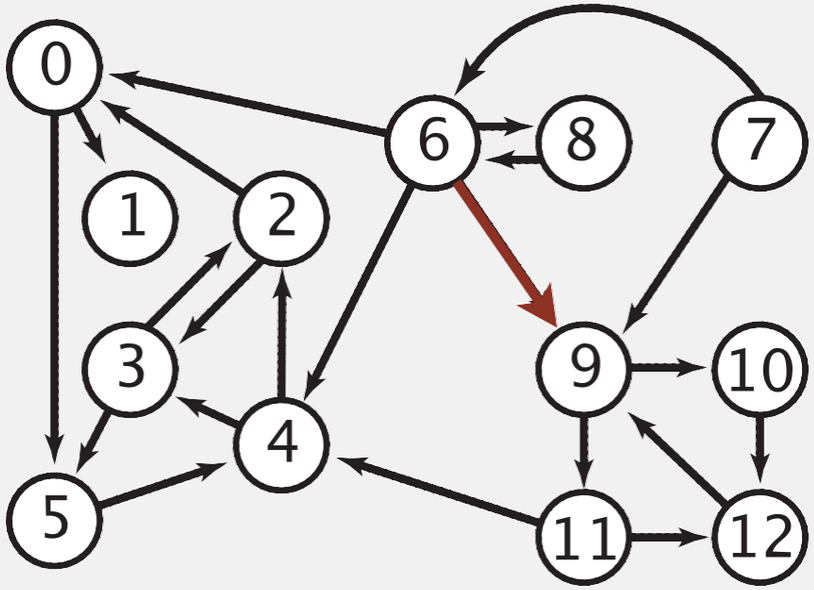
for (int v = 0; v < G.V(); v++)
   for (int w : G.adj(v))
      StdOut.println(v + "->" + w);
```

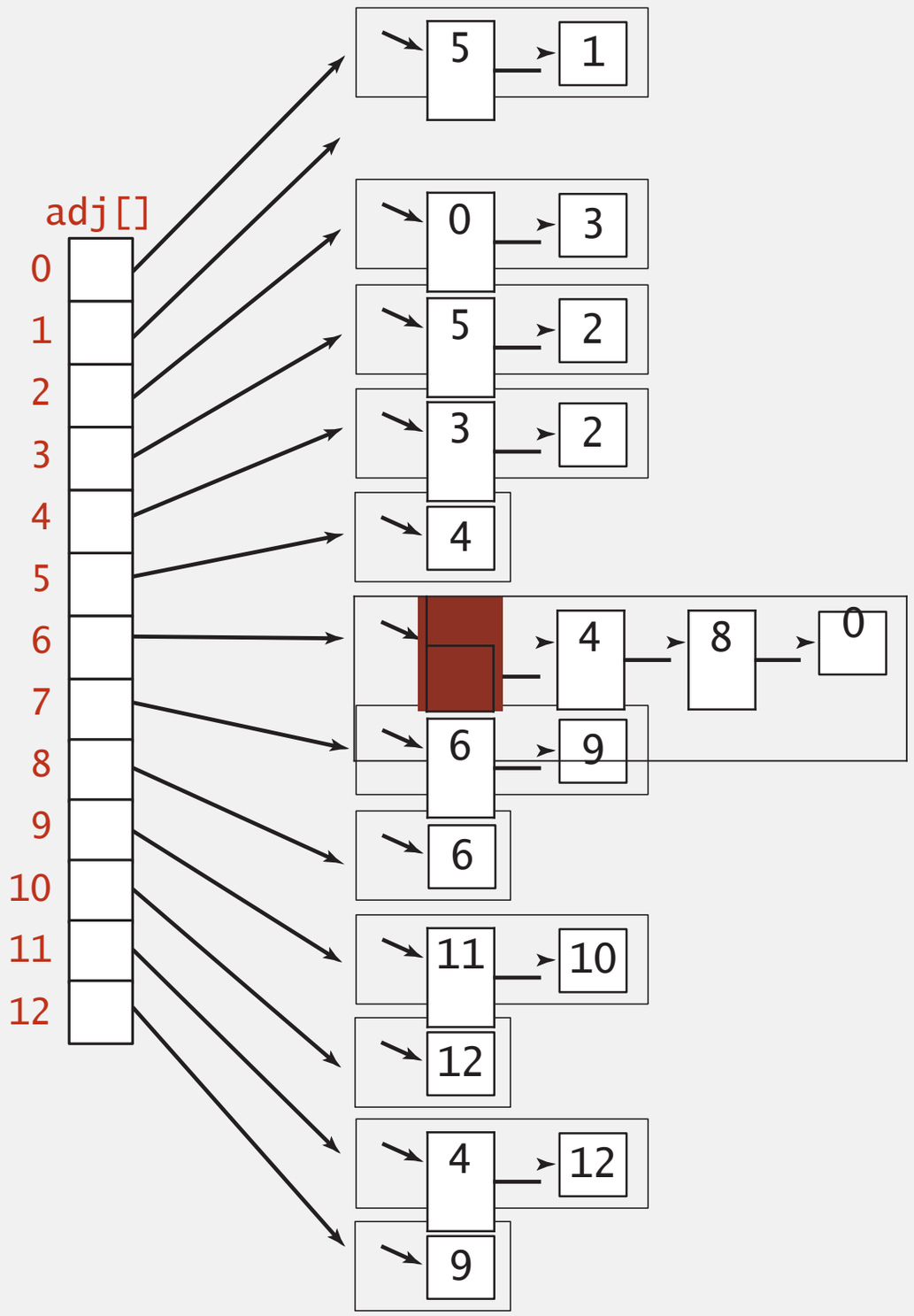
read digraph from input stream

print out each edge (once)

Paraqitja e digrafit: adjacency lists

Mirëmbaje vertex-indexed array of lists.





Paraqitja e digrafit

Në praktikë. Përdoret paraqitja adjacency-lists.

- Algoritme që iterojnë nëpër kulme që drejtohen nga pika v.
- Digrafet reale kanë tendencë të jenë sparse.

huge number of vertices, small average vertex degree

representation	space	insert edge from v to w	edge from v to w?	iterate over vertices pointing from v?
list of edges	E	1	E	E
adjacency matrix	V^2	1^*	1	V
adjacency lists	$E + V$	1	$outdegree(v)$	$outdegree(v)$

* disallows parallel edges

Adjacency-lists graph representation (review): Java implementation

```
public class Graph
{
   private final int V;
   private final Bag<Integer>[] adj;

   public Graph(int V)
   {
      this.V = V;
      adj = (Bag<Integer>[]) new Bag[V];
      for (int v = 0; v < V; v++)
         adj[v] = new Bag<Integer>();
   }

   public void addEdge(int v, int w)
   {
      adj[v].add(w);
      adj[w].add(v);
   }

   public Iterable<Integer> adj(int v)
   {  return adj[v];  }
}
```

adjacency lists

create empty graph with V vertices

add edge v-w

iterator for vertices adjacent to v

Adjacency-lists digraph representation: Java implementation

```
public class Digraph
{
   private final int V;
   private final Bag<Integer>[] adj;

   public Digraph(int V)
   {
      this.V = V;
      adj = (Bag<Integer>[]) new Bag[V];
      for (int v = 0; v < V; v++)
         adj[v] = new Bag<Integer>();
   }

   public void addEdge(int v, int w)
   {
      adj[v].add(w);

   }

   public Iterable<Integer> adj(int v)
   {  return adj[v];  }
}
```

adjacency lists

create empty digraph with V vertices

add edge v→w

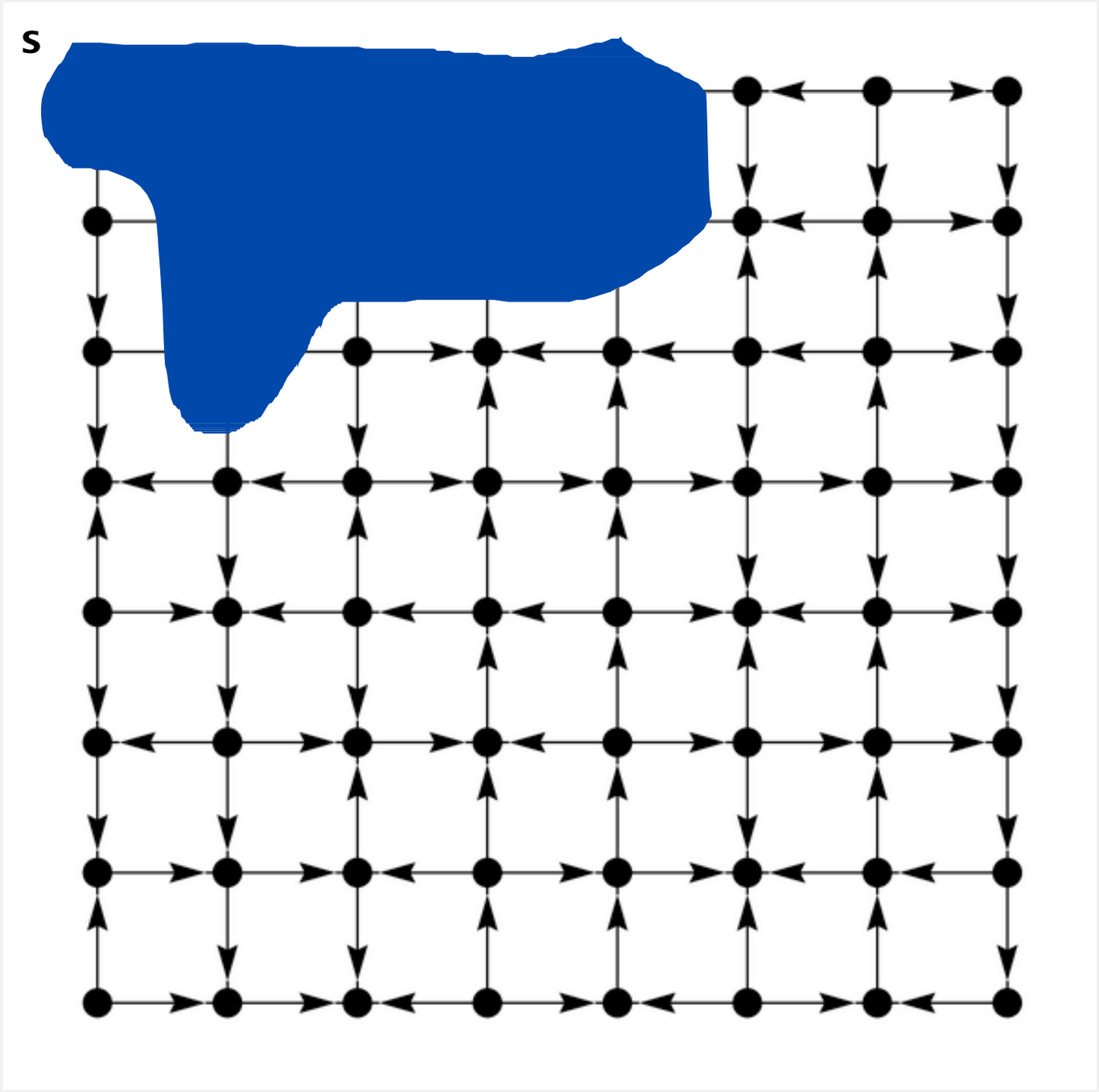
iterator for vertices pointing from v



4.2 Directed Graphs

- *introduction*
- *digraph API*
- *digraph search*
- *topological sort*
- *strong components*

Problem. Të gjinden të gjitha kulmet prej s përgjatë directed path.



Depth First Search tek digrafet

Metoda e njejtë që është perdorur për undirected graphs.

- Çdo undirected graph është një digraph (me segmente në të dyja drejtimet).
- DFS është një digraph algoritëm.

DFS (to visit a vertex v)

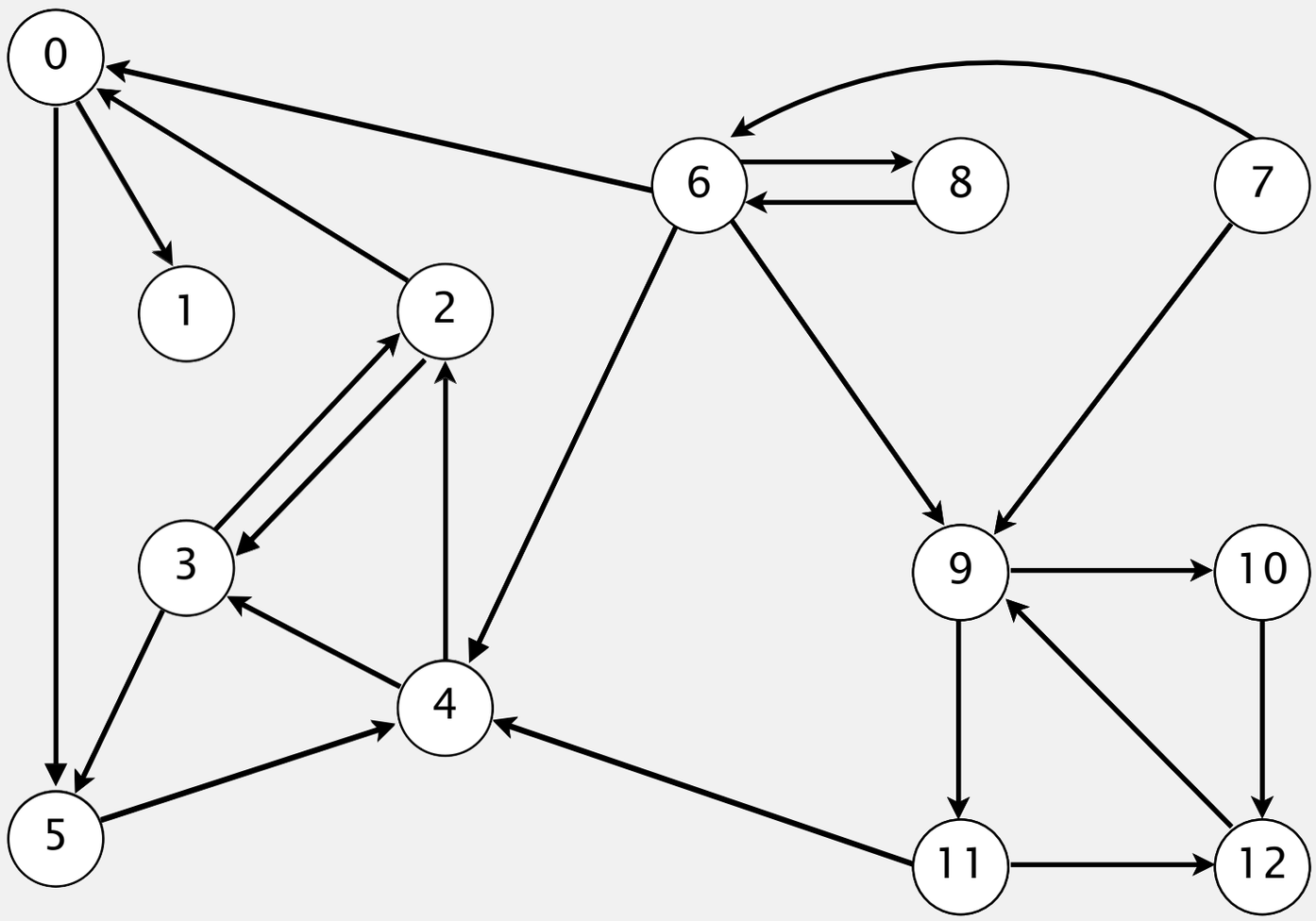
Mark v as visited.

Recursively visit all unmarked vertices w pointing from v.

Depth-first search demo

Për të vizituar një kulm v :

- Shëno kulmin v si të vizituar.
- Vazhdimisht vizito të gjitha kulmet e pashënuara që dalin nga v.



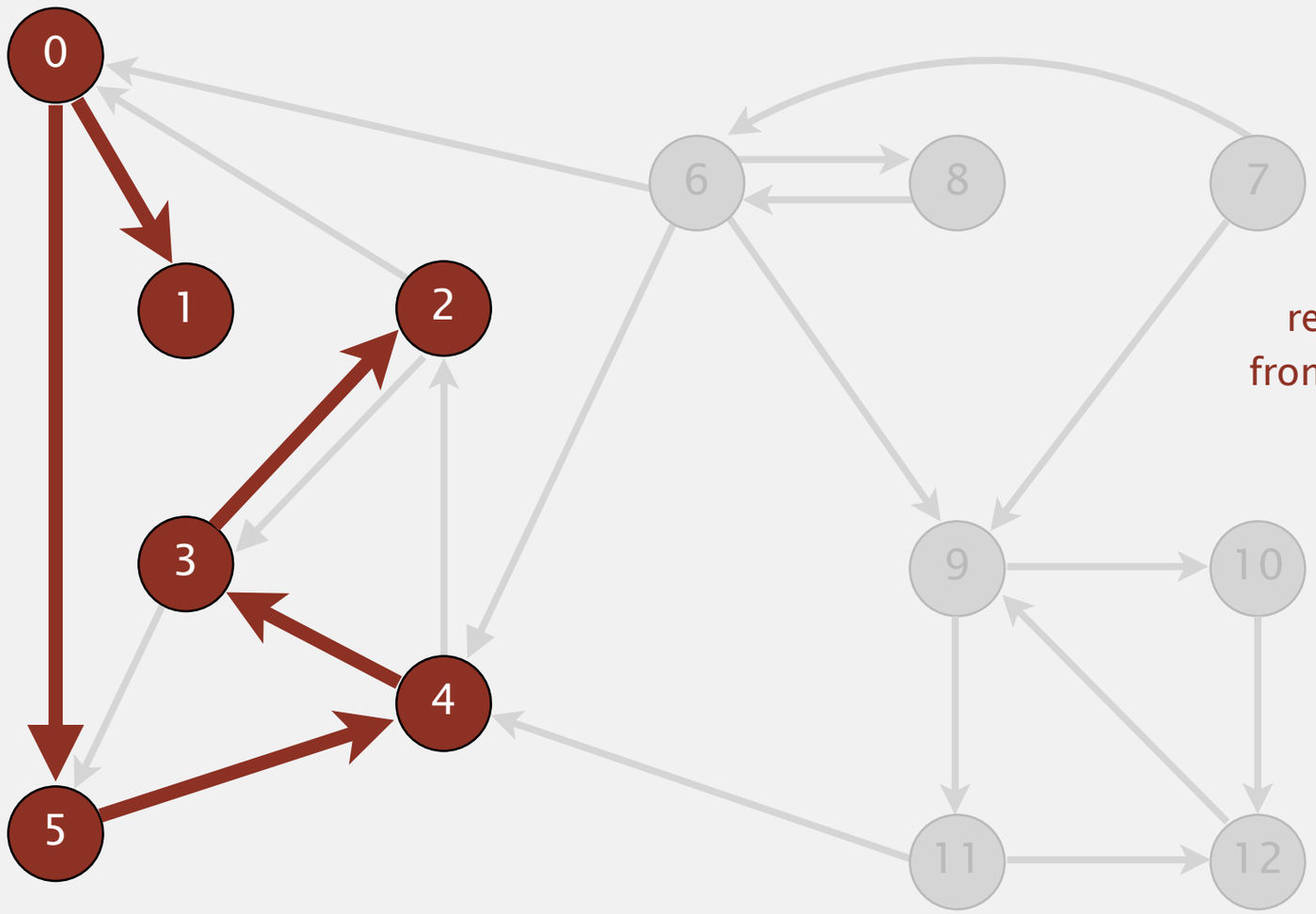
a directed graph

4→2
2→3
3→2
6→0
0→1
2→0
11→12
12→9
9→10
9→11
8→9
10→12
11→4
4→3
3→5
6→8
8→6
5→4
0→5
6→4
6→9
7→6

Depth-first search demo

Për të vizituar një kulm v :

- Shëno kulmin v si të vizituar.
- Vazhdimisht vizito të gjitha kulmet e pashënuara që dalin nga v.



reachable from vertex 0

v	marked[]	edgeTo[]
0	T	-
1	T	0
2	T	3
3	T	4
4	T	5
5	T	0
6	F	-
7	F	-
8	F	-
9	F	-
10	F	-
11	F	-
12	F	-

reachable from 0

Depth-first search (in undirected graphs)

Kodi i pasqyruar tek undirected graphs.

```
public class DepthFirstSearch
{
   private boolean[] marked;

   public DepthFirstSearch(Graph G, int s)
   {
      marked = new boolean[G.V()];
      dfs(G, s);
   }

   private void dfs(Graph G, int v)
   {
      marked[v] = true;
      for (int w : G.adj(v))
         if (!marked[w]) dfs(G, w);
   }

   public boolean visited(int v)
   {  return marked[v];  }
}
```

true if connected to s

constructor marks vertices connected to s

recursive DFS does the work

client can ask whether any vertex is connected to s

Depth-first search (in directed graphs)

Kodi për directed graphs është identik me atë të undirected graphs. [zëvendëso `Digraph` me `Graph`]

```
public class DirectedDFS
{
   private boolean[] marked;

   public DirectedDFS(Digraph G, int s)
   {
      marked = new boolean[G.V()];
      dfs(G, s);
   }

   private void dfs(Digraph G, int v)
   {
      marked[v] = true;
      for (int w : G.adj(v))
         if (!marked[w]) dfs(G, w);
   }

   public boolean visited(int v)
   {  return marked[v];  }
}
```

true if path from s

constructor marks vertices reachable from s

recursive DFS does the work

client can ask whether any vertex is reachable from s

Aplikimet - arritshmëria: program control-flow analysis

Çdo program është një digraf.

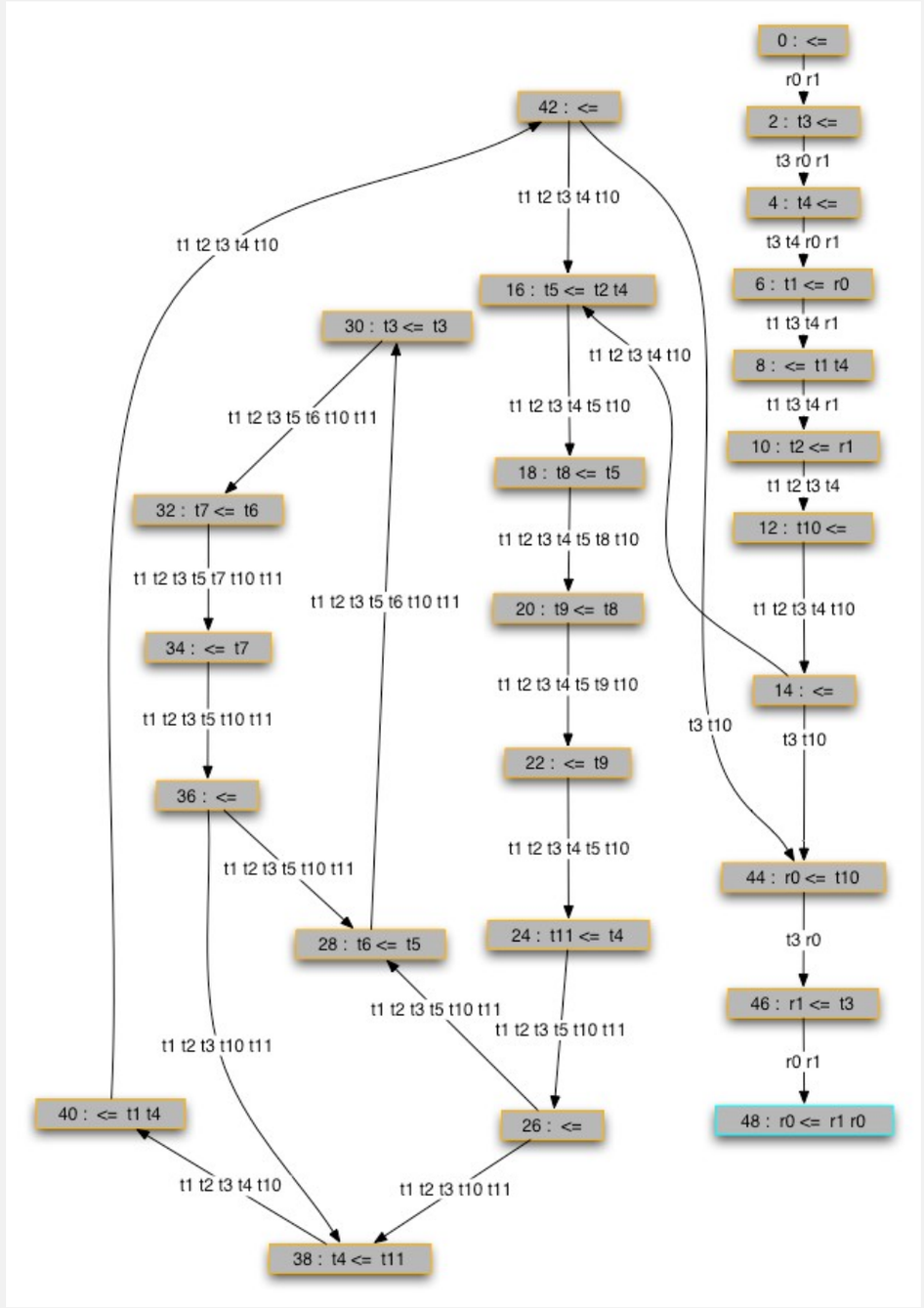
- Kulm = bllok i instrukcioneve (straight-line program).
- Segment = jump.

Eliminimi i Dead-code.

Gjen (dhe largon) kodin e paaritshëm.

Detektimi i Infinite-loop.

Përcakton a është e arritshme dalja.



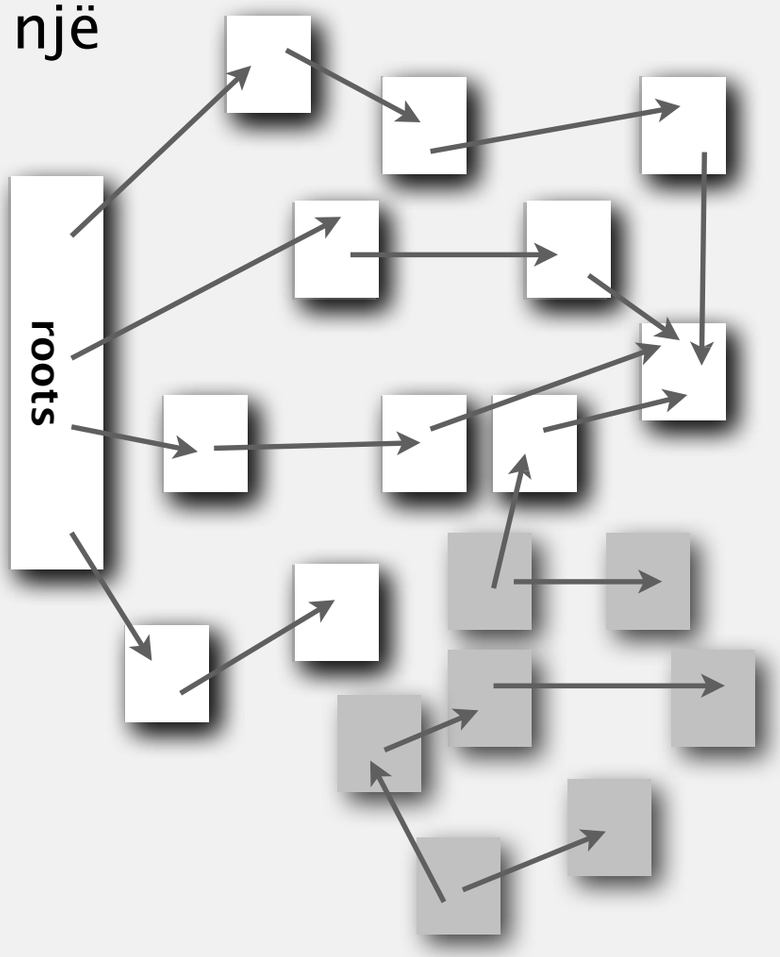
Reachability application: mark-sweep garbage collector

Çdo strukturë e të dhënave është një digraf.

- Kulmi = objekt.
- Segmenti = referencë.

Roots. Objektet që mund të qasen direct prej programit (p.sh. stack).

Reachable objects. Objektet që mund të qasen indirect prej programit (fillojnë në një root dhe përcjellin një zingjir të pointerave).

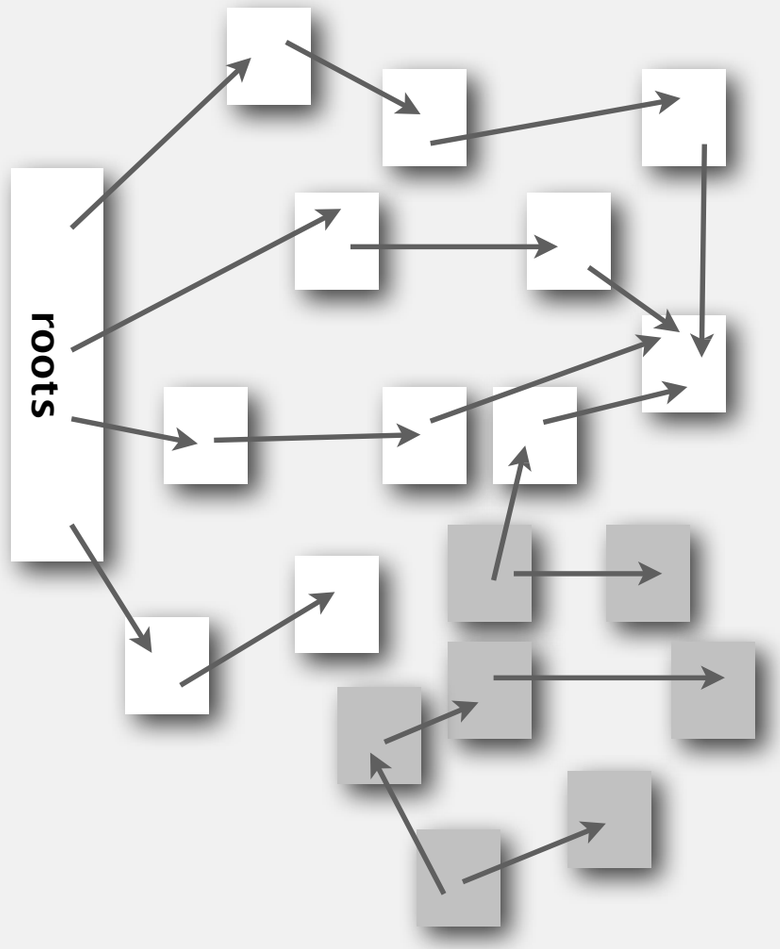


Reachability application: mark-sweep garbage collector

Mark-sweep algoritmi. [McCarthy, 1960]

- Mark: Shëno të gjitha objektet e arritshme.
- Fshij: nëse objekti është i pashënuar, paraqet mbeturina (shto në free list).

Çmimi i memories. Shpenzon një bit për shënim të objektit (plus DFS stack).



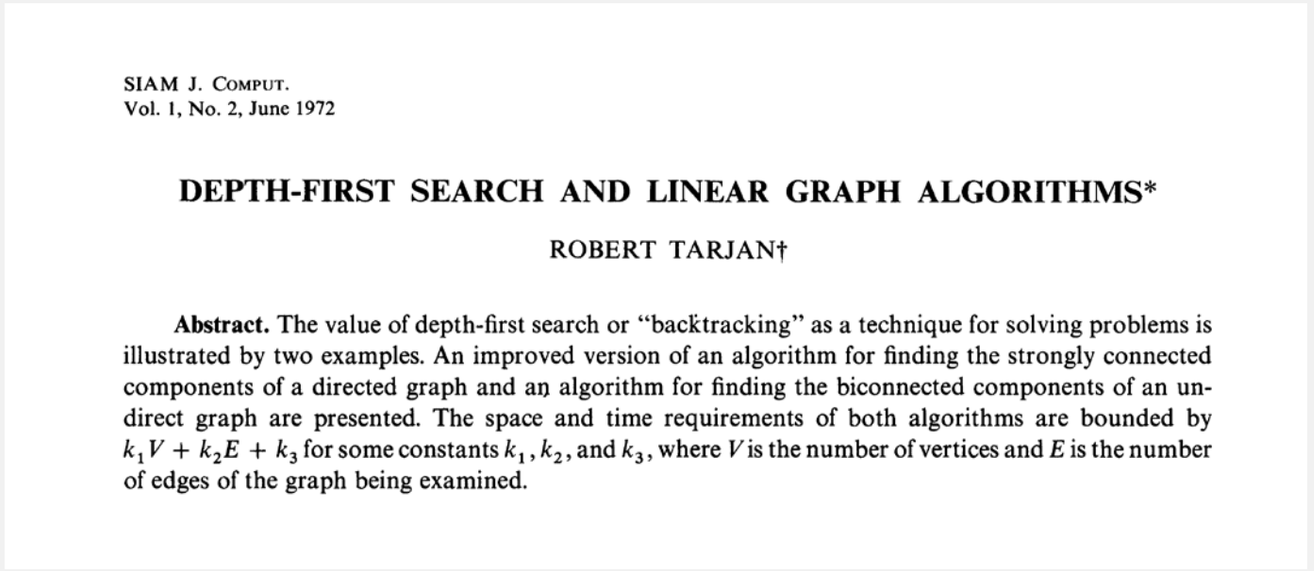
Depth-first search tek digrafet - përmbledhje

DFS enables direct solution of simple digraph problems.

- ✓ Reachability.
- Path finding.
- Topological sort.
- Detektimi i Directed cycle.

Themelet për zgjidhjen e problemeve me digraf.

- 2-satisfiability.
- Directed Euler path.
- Strongly-connected components.



SIAM J. COMPUT.
Vol. 1, No. 2, June 1972

DEPTH-FIRST SEARCH AND LINEAR GRAPH ALGORITHMS*

ROBERT TARJAN†

Abstract. The value of depth-first search or "backtracking" as a technique for solving problems is illustrated by two examples. An improved version of an algorithm for finding the strongly connected components of a directed graph and an algorithm for finding the biconnected components of an undirect graph are presented. The space and time requirements of both algorithms are bounded by $k_1V + k_2E + k_3$ for some constants k_1, k_2, and k_3, where V is the number of vertices and E is the number of edges of the graph being examined.

Breadth-first search tek digrafet

Same method as for undirected graphs.

- Çdo undirected graph është një digraph (me segmente në të dyja drejtimet).
- BFS është një digraf algoritëm.

BFS (from source vertex s)

Put s onto a FIFO queue, and mark s as visited.

Repeat until the queue is empty:

- **remove the least recently added vertex v**
- **for each unmarked vertex pointing from v:**

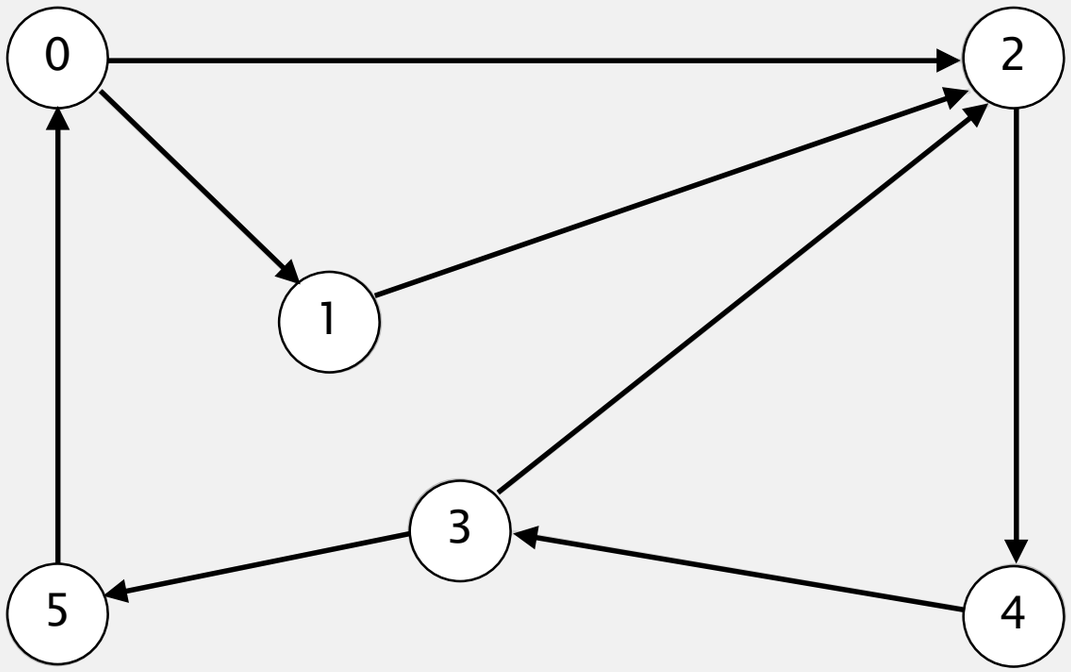
  add to queue and mark as visited.

Teoremë. BFS llogaritë shortest paths (numrin më të vogël të segmenteve) prej s deri tek të gjitha kulmet e tjera në një digraph në kohë proporcionale me $E + V$.

Directed breadth-first search demo

Përsërit deri sa të zbrazet queue:

- Remove kulmin v prej queue.
- Add në queue të gjitha kulmet e pashënuara që dalin nga v dhe shënoi.



graph G

tinyDG2.txt

V → 6

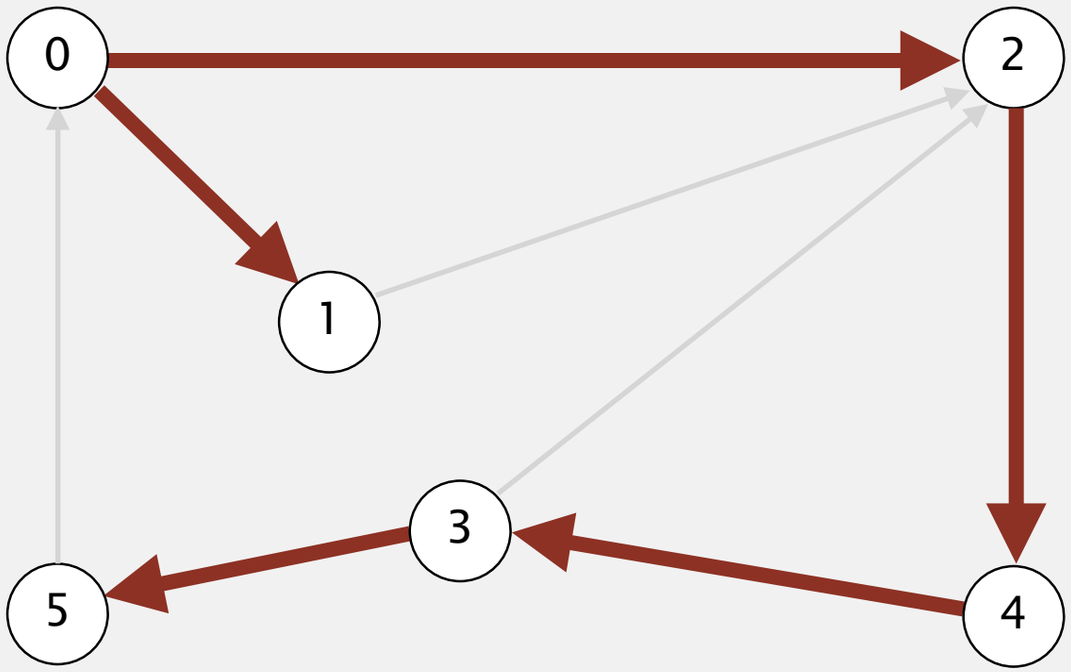
E → 8

5 0
2 4
3 2
1 2
0 1
4 3
3 5
0 2

Directed breadth-first search demo

Përsërit deri sa të zbrazet queue:

- Remove kulmin v prej queue.
- Add në queue të gjithakulmet e pashënuara që dalin nga v dhe shënoi.



v	edgeTo[]	distTo[]
0	-	0
1	0	1
2	0	1
3	4	3
4	2	2
5	3	4

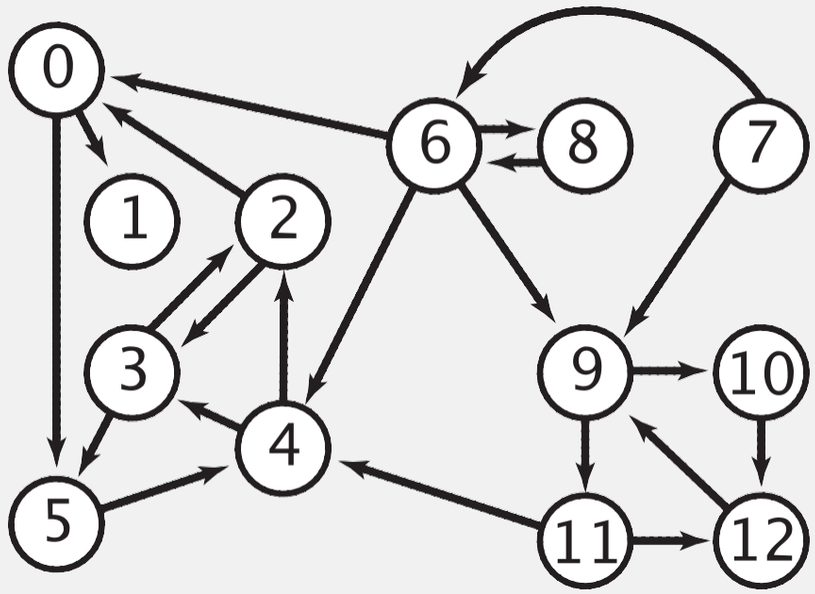
done

Multiple-source shortest paths

Multiple-source shortest paths. Le të jetë një digraf bashkësi kulmeve burimore, gjej shortest path prej cilido kulm në bashkësi tek secili kulm tjeter.

Ex. $S = \{ 1, 7, 10 \}$.

- Shortest path to 4 is $7 \rightarrow 6 \rightarrow 4$.
- Shortest path to 5 is $7 \rightarrow 6 \rightarrow 0 \rightarrow 5$.
- Shortest path to 12 is $10 \rightarrow 12$.
- …



Q. How to implement multi-source shortest paths algorithm?

A. Use BFS, but initialize by enqueuing all source vertices.



4.2 Directed Graphs

- *introduction*
- *digraph API*
- *digraph search*
- *topological sort*
- *strong components*

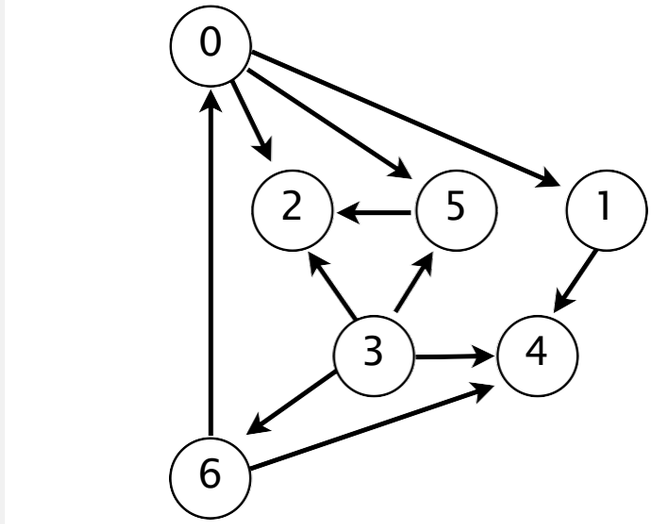
Precedence scheduling

Qëllimi. Le të jetë një bashkësi me detyra që duhet kryer me precedence constraints. Në çfarë rradhitje duhet të jenë detyrat?

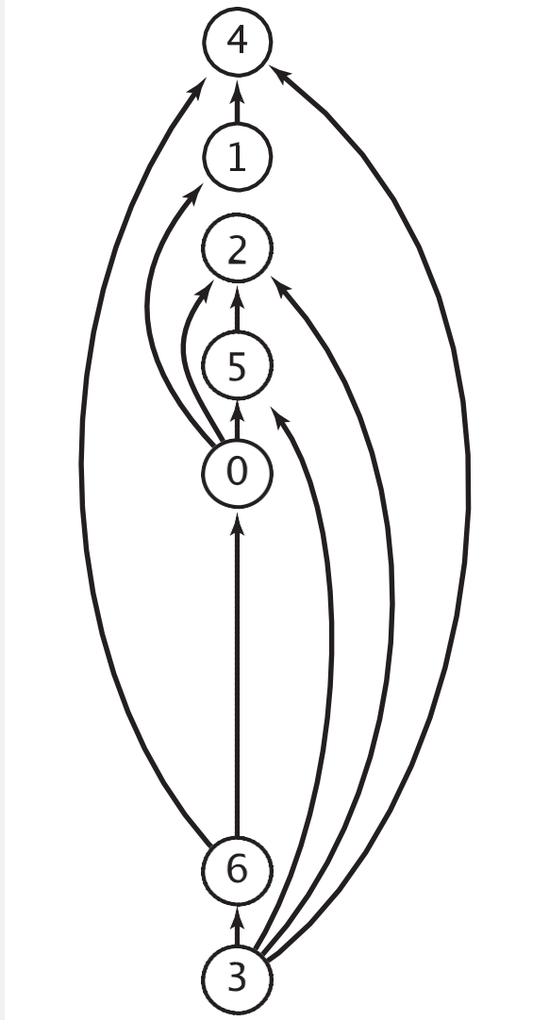
Digraf modeli. kulmi = detyra; segmenti = precedence constraint.

1. Algorithms
2. Complexity Theory
3. Artificial Intelligence
4. Intro to CS
5. Cryptography
6. Scientific Computing
7. Advanced Programming

tasks



precedence constraint graph



feasible schedule

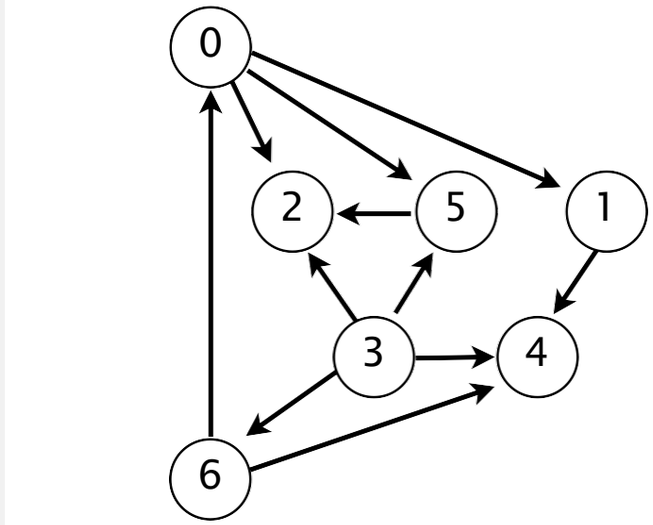
Topological sort

DAG. Directed acyclic graph.

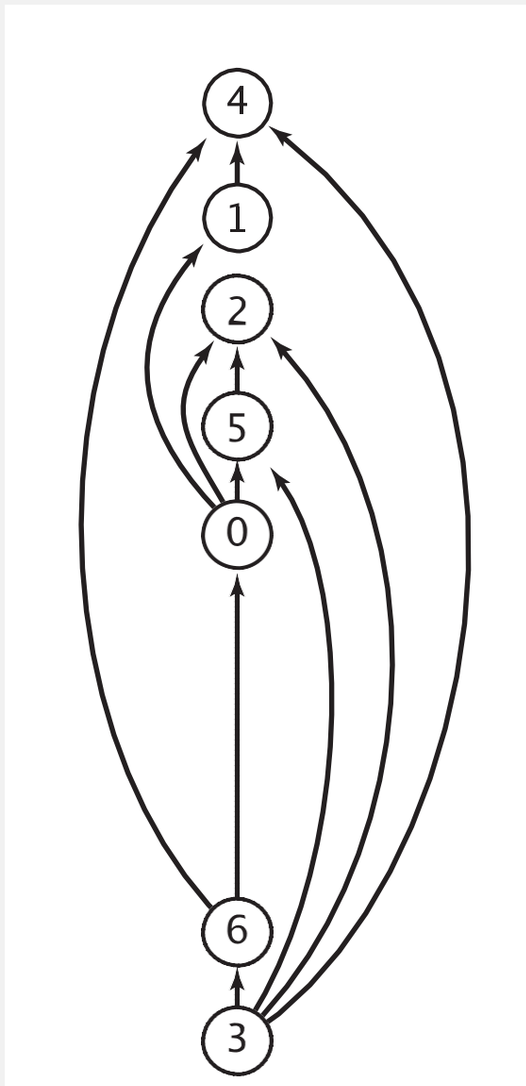
Topological sort. Rivizato DAG ashtu që të gjitha segmented tregojnë lartë.

0→5	0→2
0→1	3→6
3→5	3→4
5→2	6→4
6→0	3→2
1→4	

directed edges



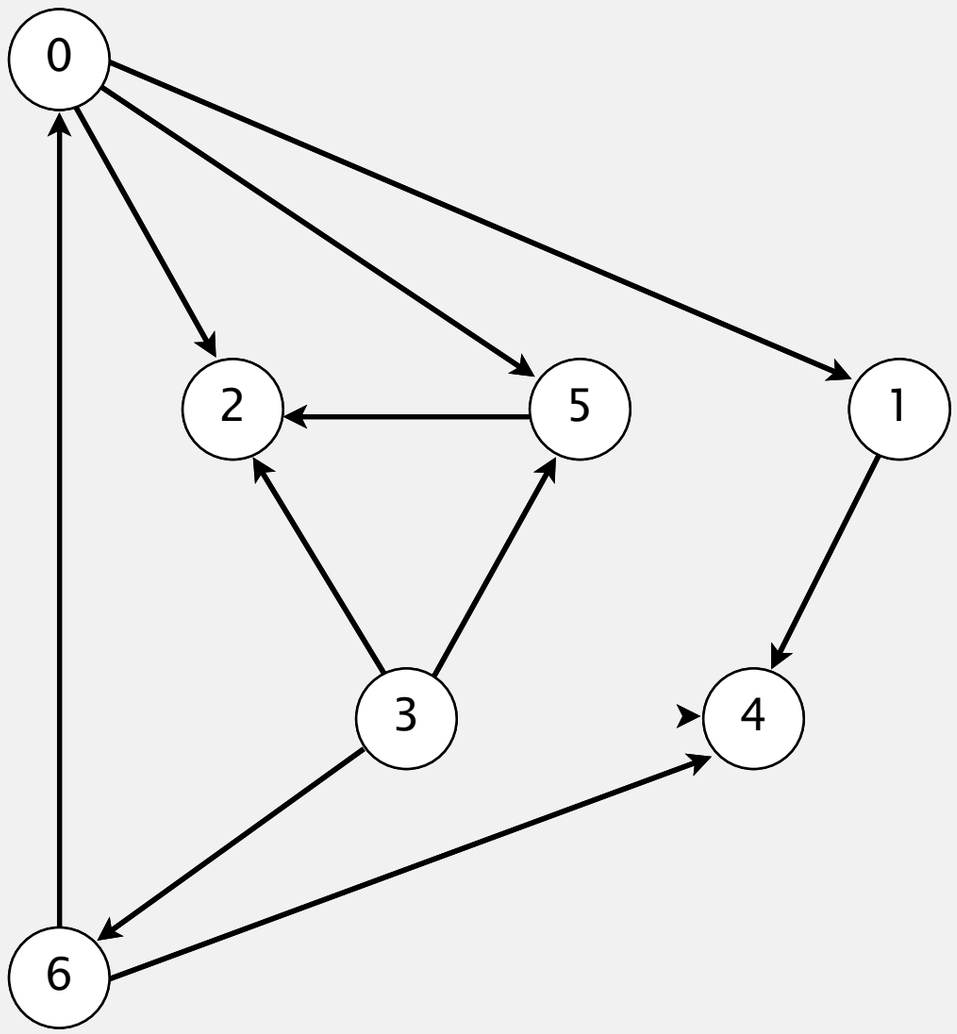
DAG



topological order

Topological sort demo

- Run depth-first search.
- Return vertices in reverse postorder.



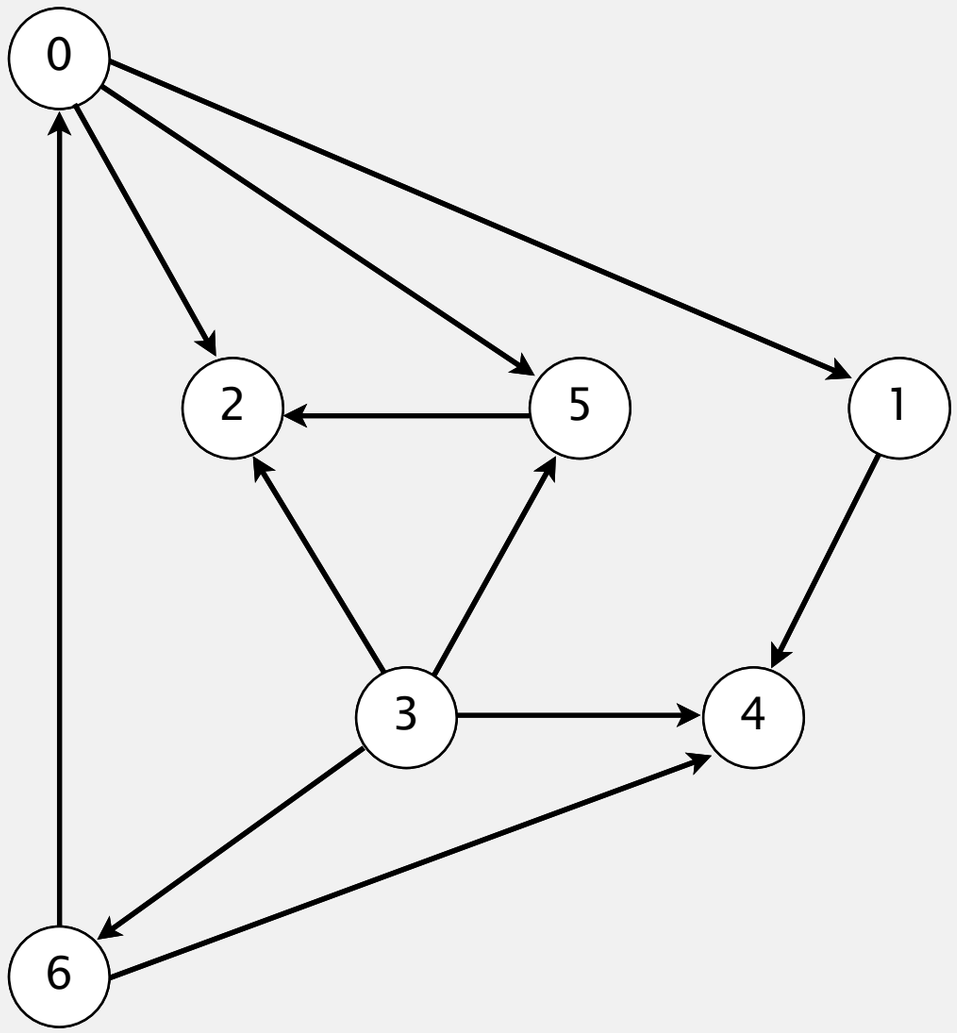
tinyDAG7.txt

```
7
11
 0  5
 0  2
 0  1
 3  6
 3  5
 3  4
 5  2
 6  4
 6  0
 3  2
```

a directed acyclic graph

Topological sort demo

- Run depth-first search.
- Return vertices in reverse postorder.



postorder

4 1 2 5 0 6 3

topological order

3 6 0 5 2 1 4

done

Depth-first search order

```
public class DepthFirstOrder
{
   private boolean[] marked;
   private Stack<Integer> reversePostorder;

   public DepthFirstOrder(Digraph G)
   {
      reversePostorder = new Stack<Integer>();
      marked = new boolean[G.V()];
      for (int v = 0; v < G.V(); v++)
         if (!marked[v]) dfs(G, v);
   }

   private void dfs(Digraph G, int v)
   {
      marked[v] = true;
      for (int w : G.adj(v))
         if (!marked[w]) dfs(G, w);
      reversePostorder.push(v);
   }

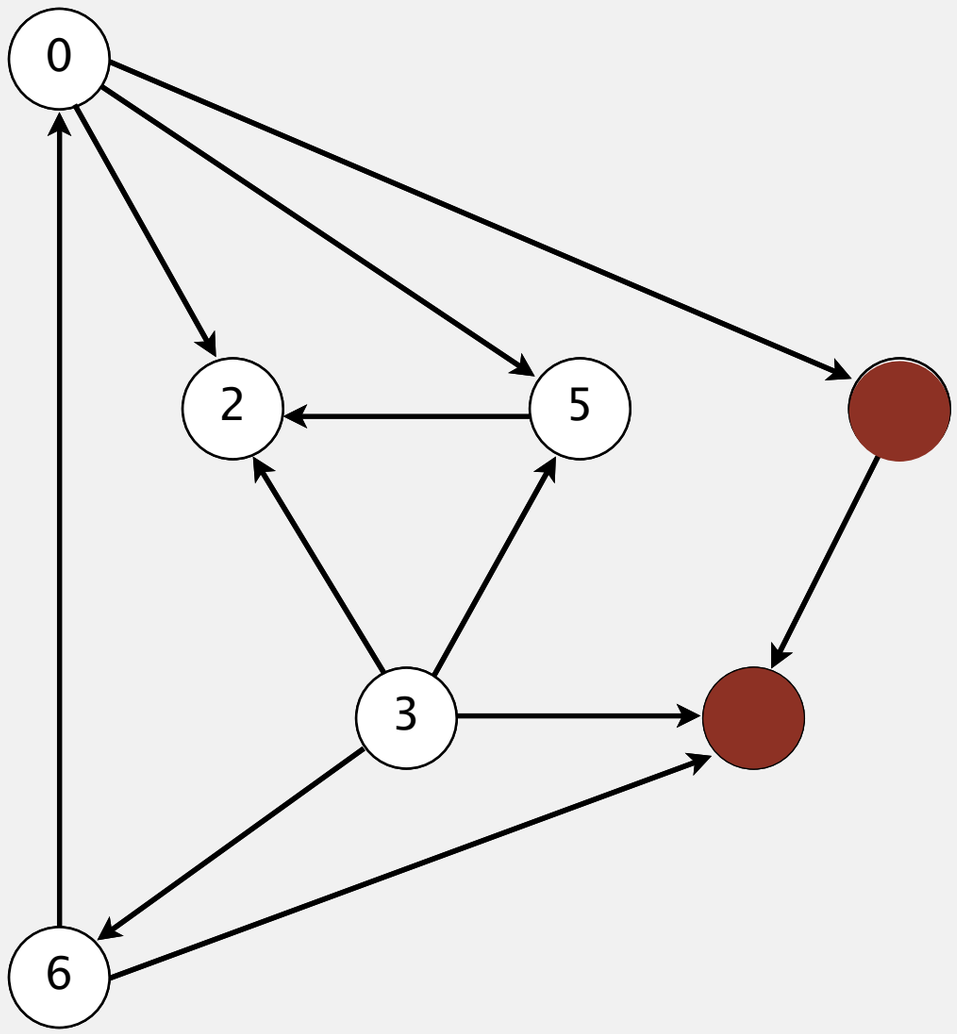
   public Iterable<Integer> reversePostorder()
   {  return reversePostorder;  }
}
```

returns all vertices in "reverse DFS postorder"

Topological sort in a DAG: intuition

Why does topological sort algorithm work?

- First vertex in postorder has outdegree 0.
- Second-to-last vertex in postorder can only point to last vertex.
- ...



postorder

4 1 2 5 0 6 3

topological order

3 6 0 5 2 1 4

Topological sort in a DAG: correctness proof

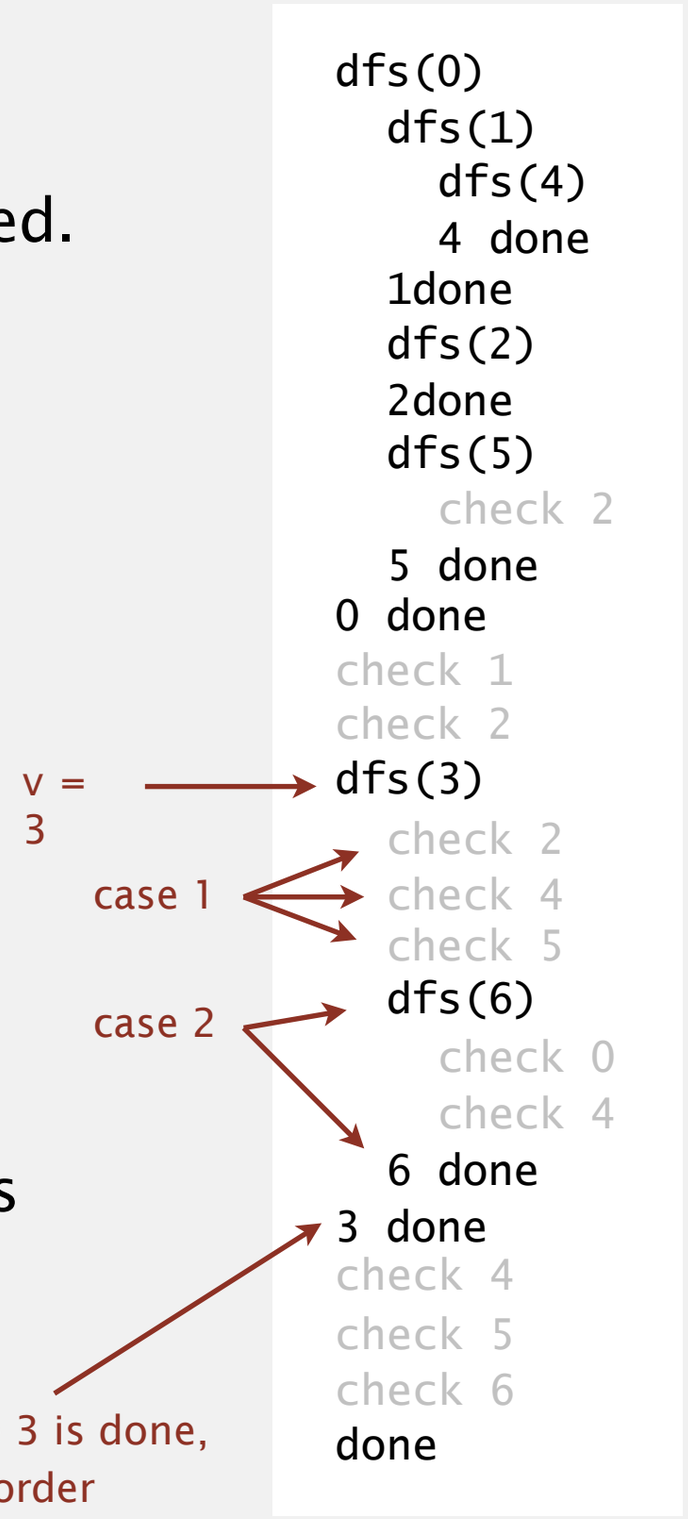
Proposition. Reverse DFS postorder of a DAG is a topological order.

Pf. Consider any edge v→w. When dfs(v) is called:

- Case 1: dfs(w) has already been called and returned.
  Thus, w was done before v.

- Case 2: dfs(w) has not yet been called.
  dfs(w) will get called directly or indirectly
  by dfs(v) and will finish before dfs(v).
  Thus, w will be done before v.

- Case 3: dfs(w) has already been called,
  but has not yet returned.
  Can't happen in a DAG: function call stack contains
  path from w to v, so v→w would complete a cycle.

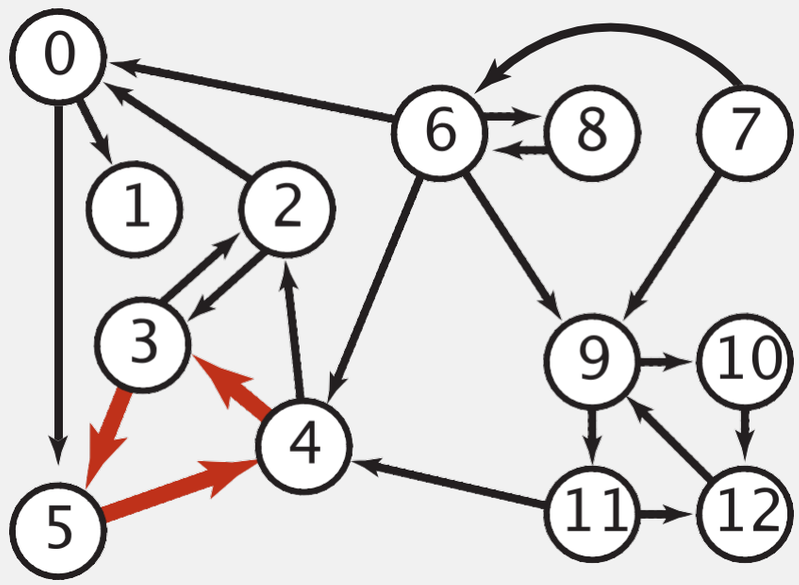


Directed cycle detection

Proposition. A digraph has a topological order iff no directed cycle.

Pf.

- If directed cycle, topological order impossible.
- If no directed cycle, DFS-based algorithm finds a topological order.



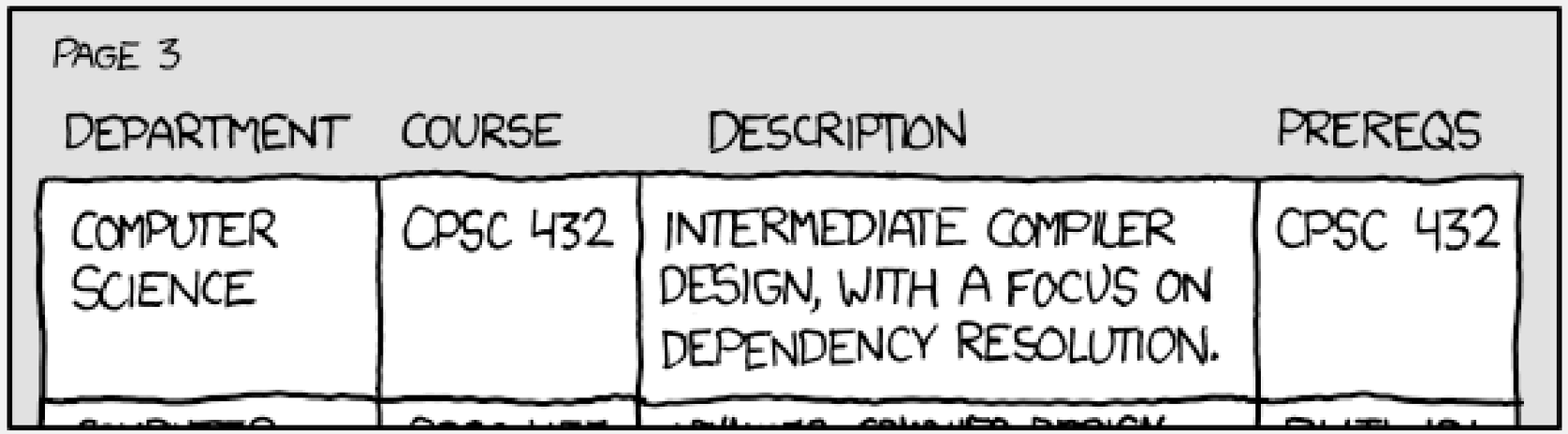
a digraph with a directed cycle

Goal. Given a digraph, find a directed cycle.

Solution. DFS. What else? See textbook.

Directed cycle detection application: precedence scheduling

Scheduling. Given a set of tasks to be completed with precedence constraints, in what order should we schedule the tasks?



http://xkcd.com/754

Remark. A directed cycle implies scheduling problem is infeasible.

Directed cycle detection application: cyclic inheritance

The Java compiler does cycle detection.

```
public class A extends B
{
   ...
}
```

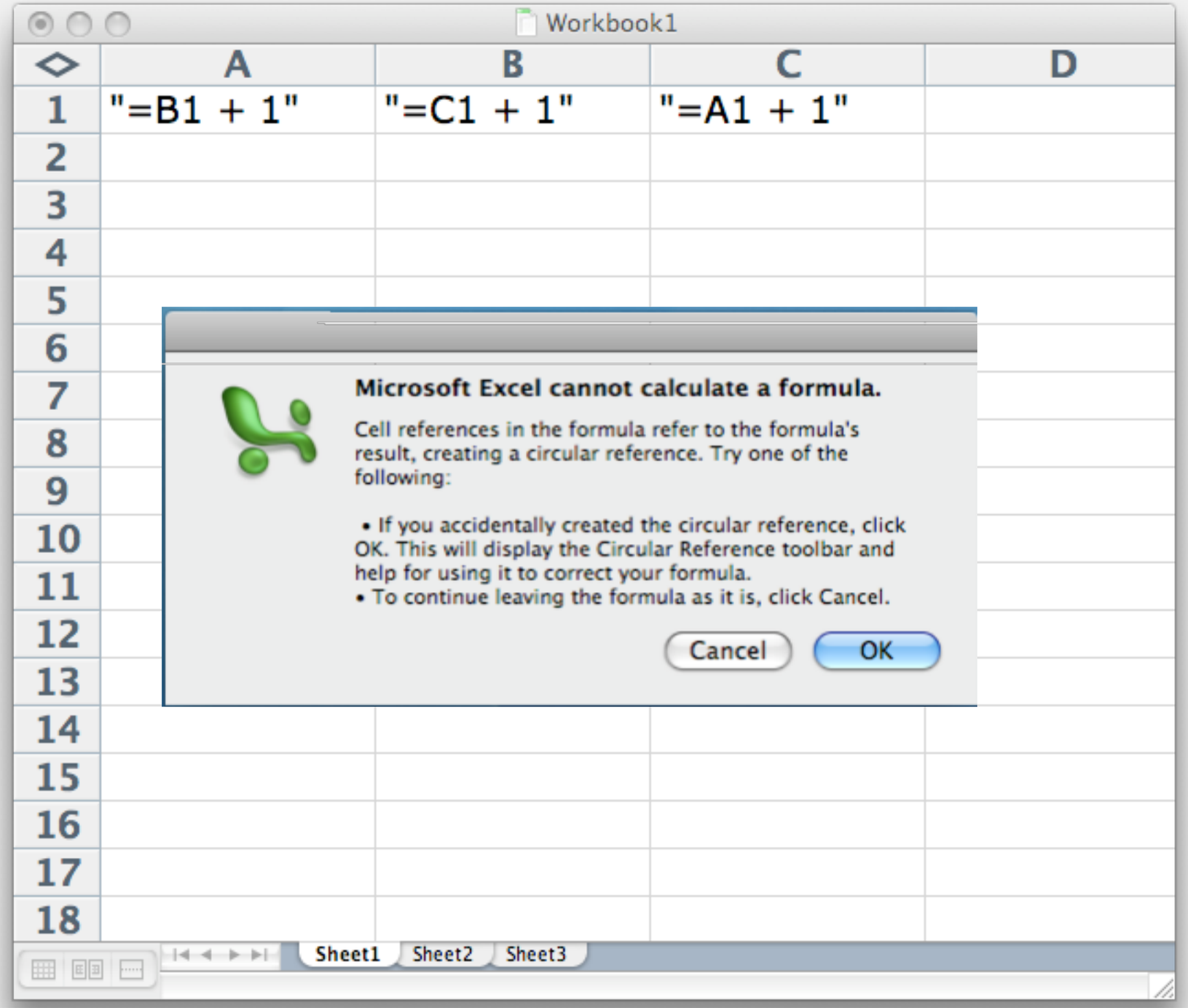
```
public class B extends C
{
   ...
}
```

```
public class C extends A
{
   ...
}
```

```
% javac A.java
A.java:1: cyclic inheritance
involving A
public class A extends B { }
       ^
1 error
```

Directed cycle detection application: spreadsheet recalculation

Microsoft Excel does cycle detection (and has a circular reference toolbar!)



Depth-first search orders

Observation. DFS visits each vertex exactly once. The order in which it does so can be important.

Orderings.

- Preorder: order in which `dfs()` is called.
- Postorder: order in which `dfs()` returns.
- Reverse postorder: reverse order in which `dfs()` returns.

```
private void dfs(Graph G, int v)
{
   marked[v] = true;
   preorder.enqueue(v);
   for (int w : G.adj(v))
      if (!marked[w]) dfs(G, w);
   postorder.enqueue(v);
   reversePostorder.push(v);
}
```



4.2 Directed Graphs

- *introduction*
- *digraph API*
- *digraph search*
- *topological sort*
- *strong components*

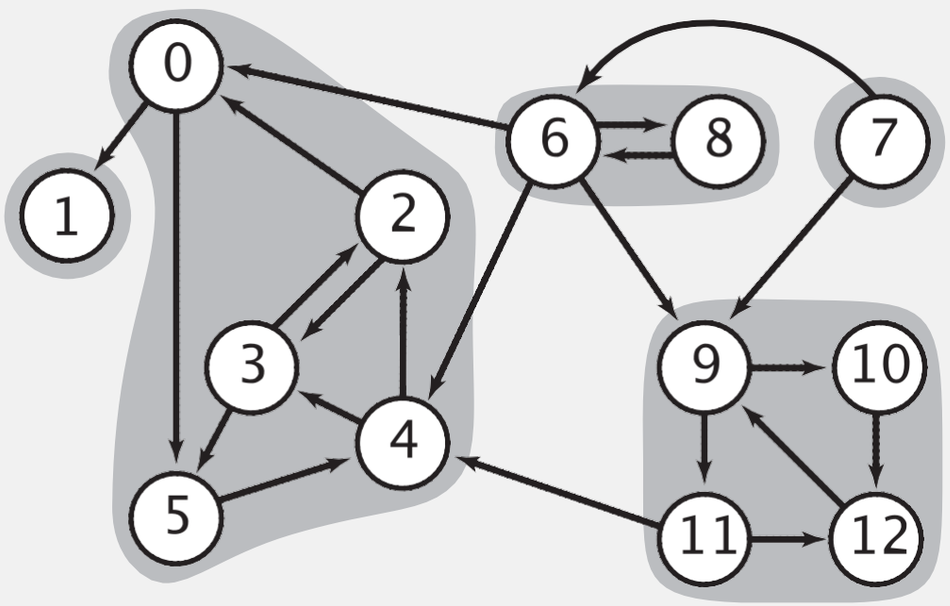
Strongly-connected components

Def. Vertices v and w are strongly connected if there is both a directed path from v to w and a directed path from w to v.

Key property. Strong connectivity is an equivalence relation:

- v is strongly connected to v.
- If v is strongly connected to w, then w is strongly connected to v.
- If v is strongly connected to w and w to x, then v is strongly connected to x.

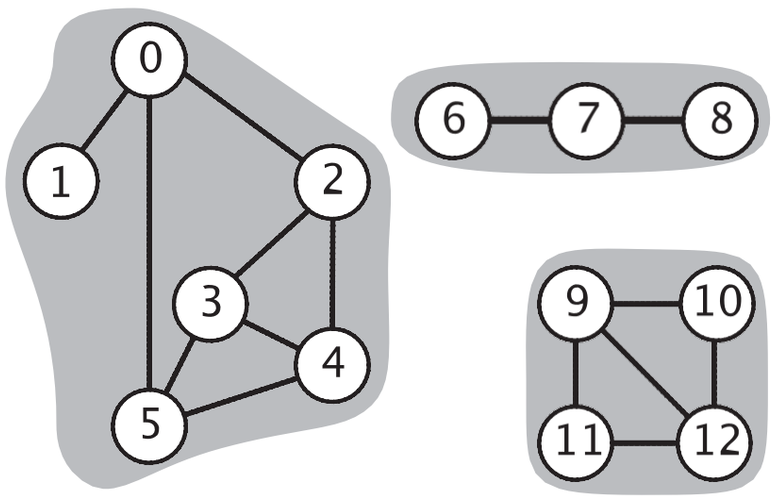
Def. A strong component is a maximal subset of strongly-connected vertices.



5 strongly-connected components

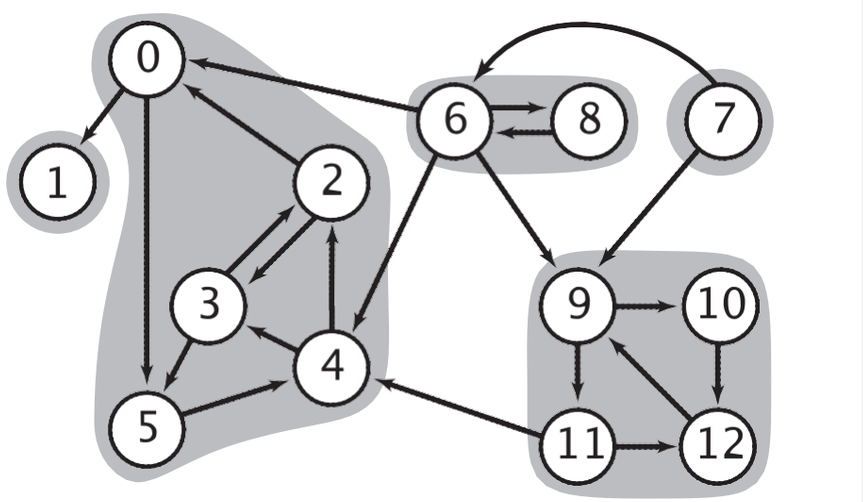
Connected components vs. strongly-connected components

v and w are connected if there is a path between v and w



3 connected components

v and w are strongly connected if there is both a directed path from v to w and a directed path from w to v



5 strongly-connected components

connected component id (easy to compute with DFS)

	0	1	2	3	4	5	6	7	8	9	10	11	12
id[]	0	0	0	0	0	0	1	1	1	2	2	2	2

```
public boolean connected(int v, int w)
{  return id[v] == id[w];  }
```

constant-time client connectivity query

strongly-connected component id (how to compute?)

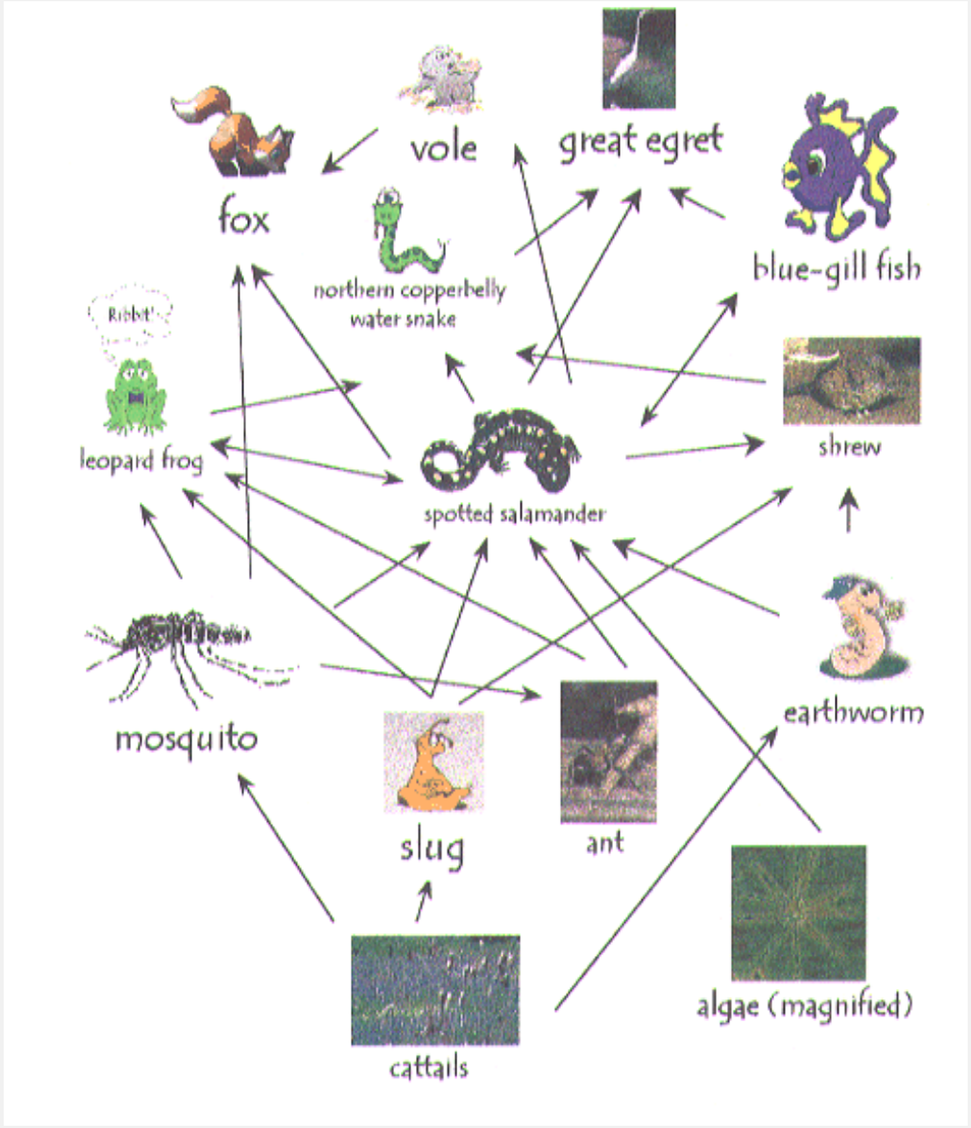
	0	1	2	3	4	5	6	7	8	9	10	11	12
id[]	1	0	1	1	1	1	3	4	3	2	2	2	2

```
public boolean stronglyConnected(int v, int w)
{  return id[v] == id[w];  }
```

constant-time client strong-connectivity query

Strong component application: ecological food webs

Food web graph. Vertex = species; edge = from producer to consumer.



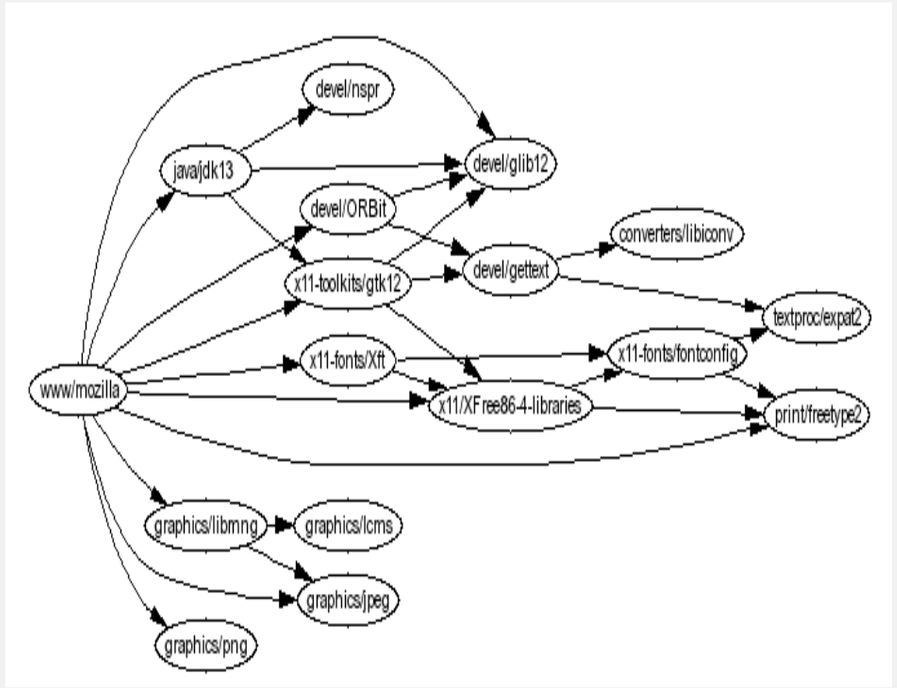
http://www.twingroves.district96.k12.il.us/Wetlands/Salamander/SalGraphics/salfoodweb.gif

Strong component. Subset of species with common energy flow.

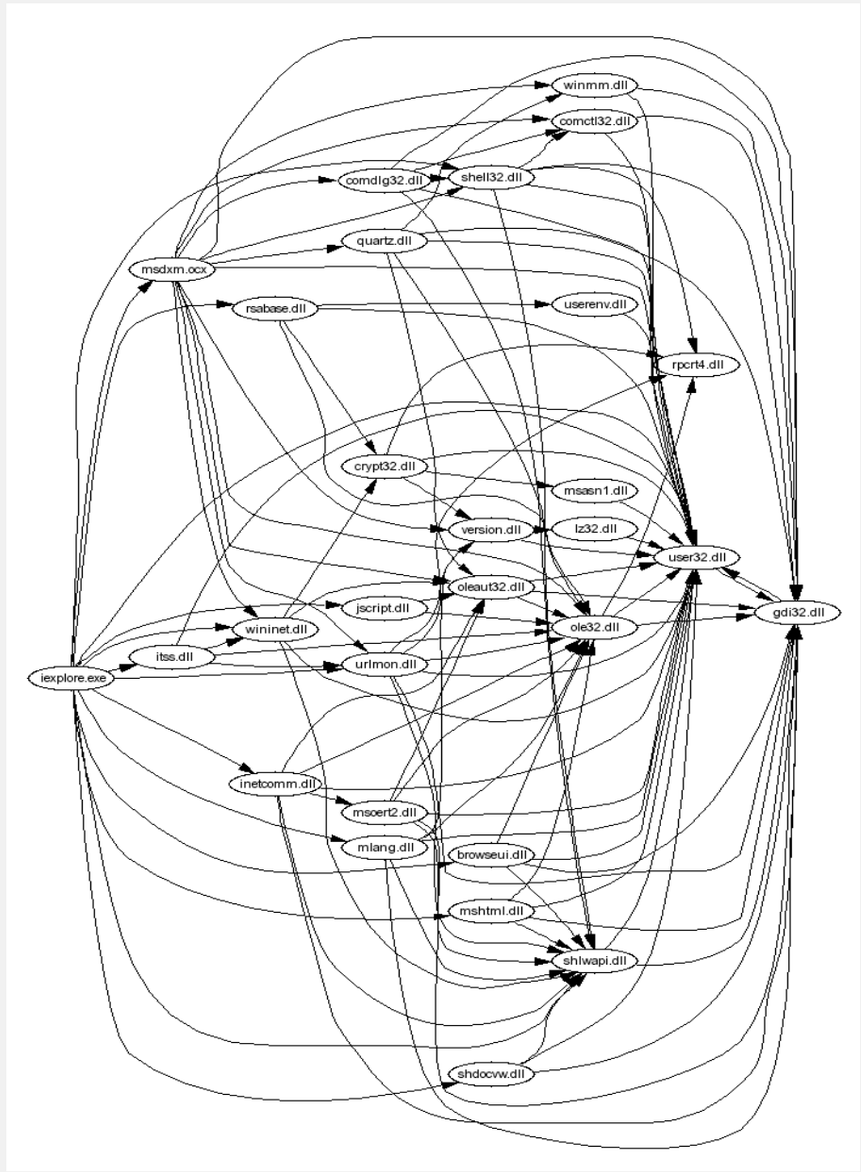
Strong component application: software modules

Software module dependency graph.

- Vertex = software module.
- Edge: from module to dependency.



Firefox



Internet Explorer

Strong component. Subset of mutually interacting modules.

Approach 1. Package strong components together.

Approach 2. Use to improve design!

Strong components algorithms: brief history

1960s: Core OR problem.

- Widely studied; some practical algorithms.
- Complexity not understood.

1972: linear-time DFS algorithm (Tarjan).

- Classic algorithm.
- Level of difficulty: Algs4++.
- Demonstrated broad applicability and importance of DFS.

1980s: easy two-pass linear-time algorithm (Kosaraju-Sharir).

- Forgot notes for lecture; developed algorithm in order to teach it!
- Later found in Russian scientific literature (1972).

1990s: more easy linear-time algorithms.

- Gabow: fixed old OR algorithm.
- Cheriyan-Mehlhorn: needed one-pass algorithm for LEDA.

Kosaraju-Sharir algorithm: intuition

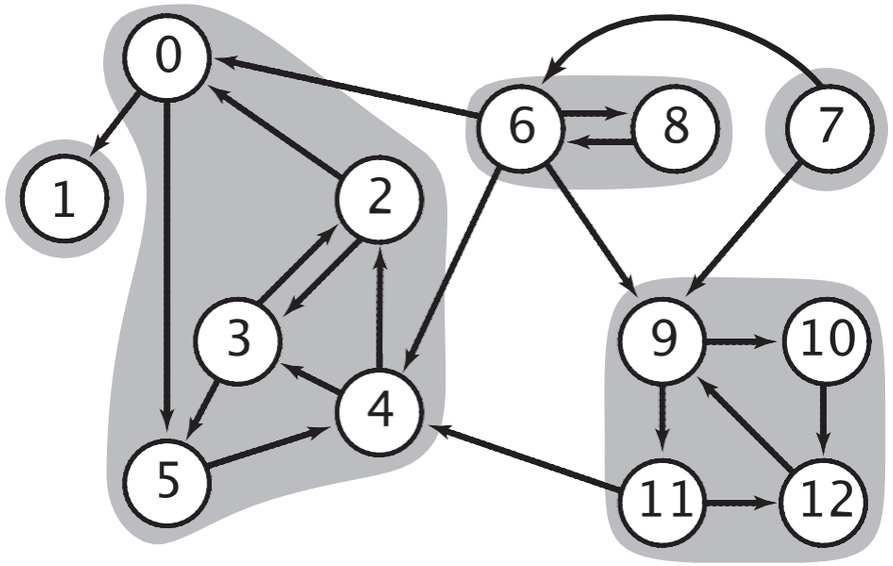
Reverse graph. Strong components in G are same as in G^R.

Kernel DAG. Contract each strong component into a single vertex.

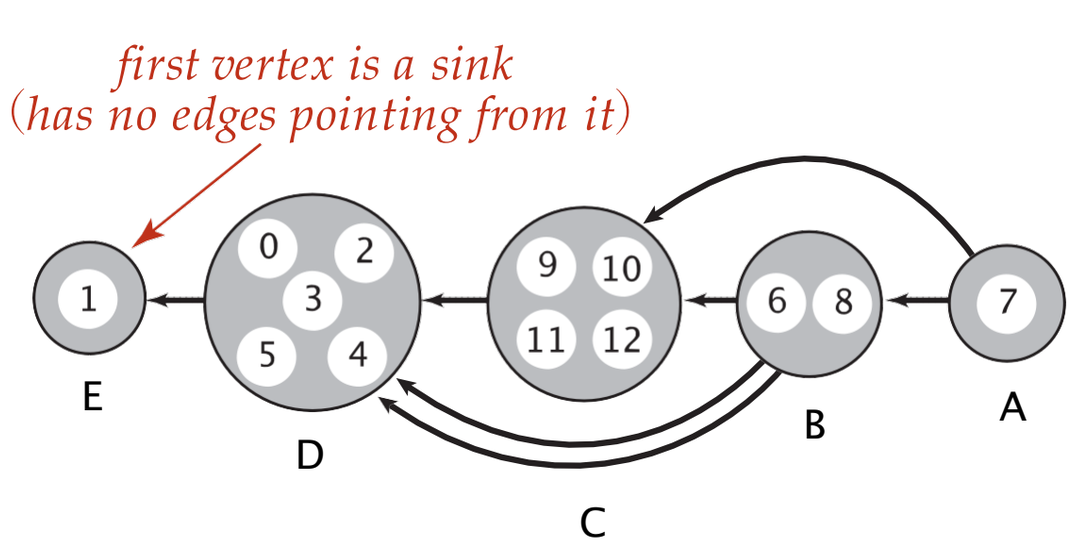
Idea.

how to compute?

- Compute topological order (reverse postorder) in kernel DAG.
- Run DFS, considering vertices in reverse topological order.



digraph G and its strong components

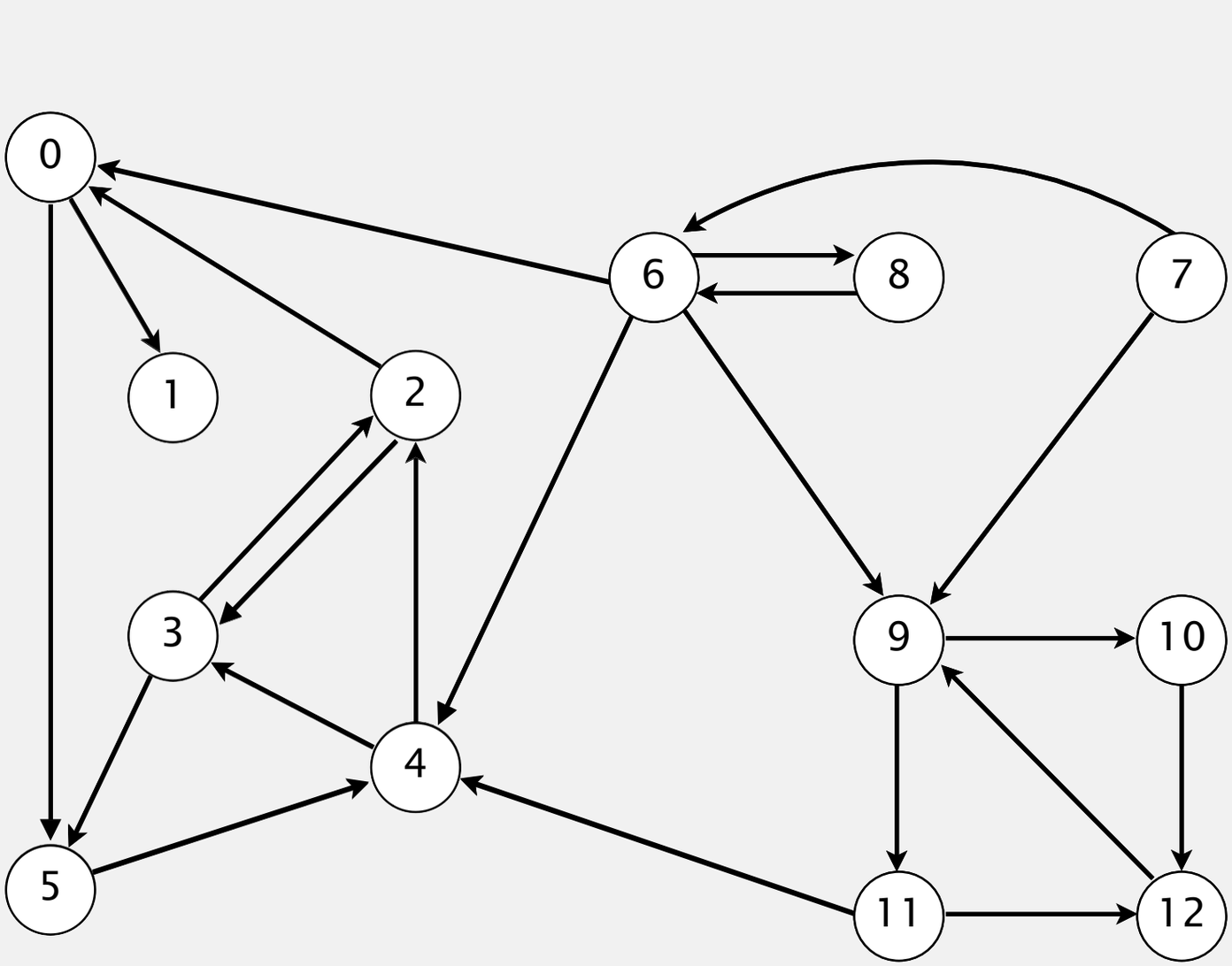


kernel DAG of G (topological order: A B C D E)

Kosaraju-Sharir algorithm demo

Phase 1. Compute reverse postorder in G^R.

Phase 2. Run DFS in G, visiting unmarked vertices in reverse postorder of G^R.

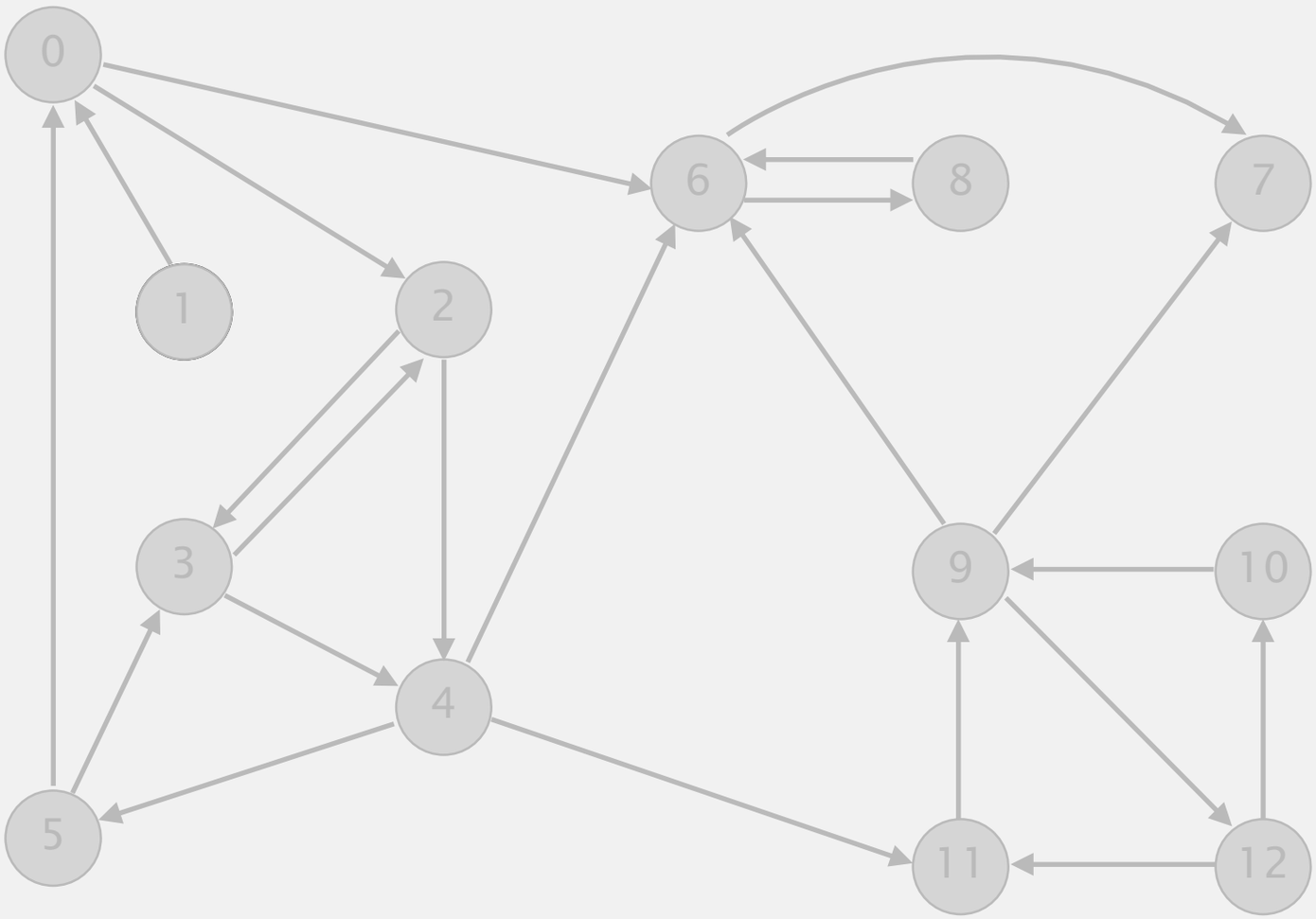


digraph G

Kosaraju-Sharir algorithm demo

Phase 1. Compute reverse postorder in G^R.

1 0 2 4 5 3 11 9 12 10 6 7 8

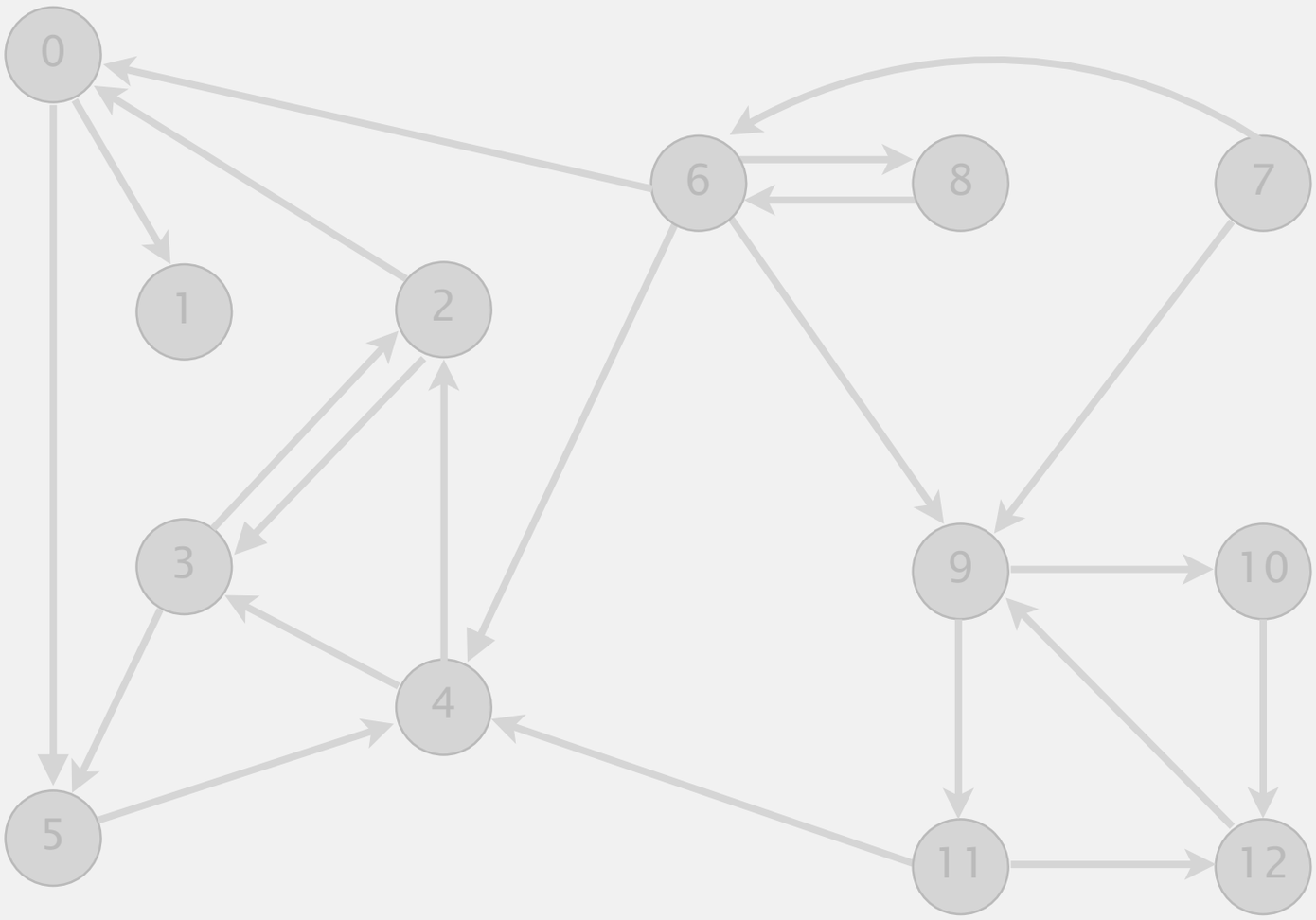


reverse digraph G^R

Kosaraju-Sharir algorithm demo

Phase 2. Run DFS in G, visiting unmarked vertices in reverse postorder of G^R.

1 0 2 4 5 3 11 9 12 10 6 7 8



v	id[]
0	1
1	0
2	1
3	1
4	1
5	1
6	3
7	4
8	3
9	2
10	2
11	2
12	2

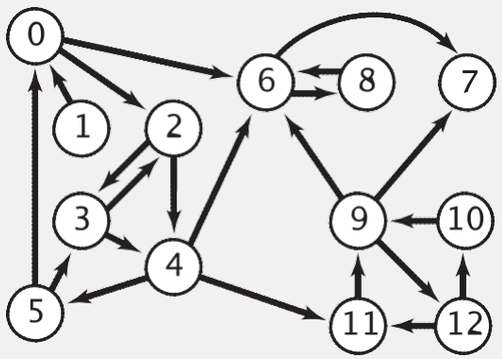
done

Kosaraju-Sharir algorithm

Simple (but mysterious) algorithm for computing strong components.

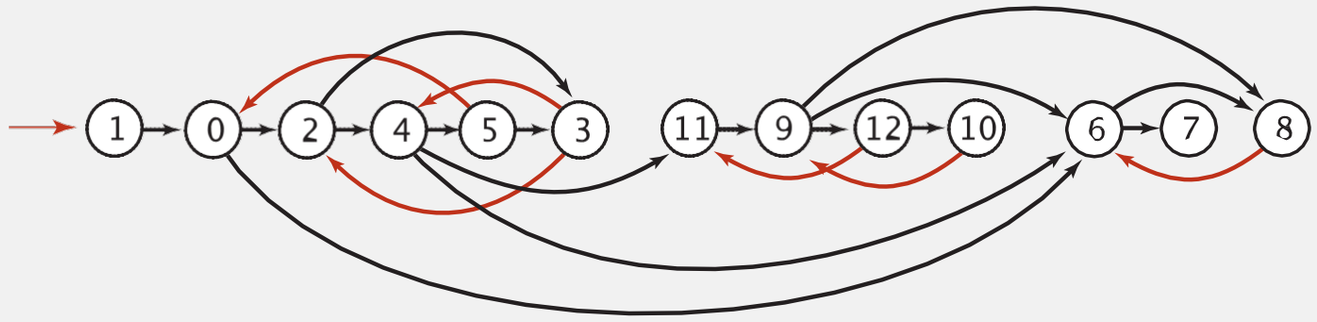
- Phase 1: run DFS on G^R to compute reverse postorder.
- Phase 2: run DFS on G, considering vertices in order given by first DFS.

DFS in reverse digraph G^R



check unmarked vertices in the order

0 1 2 3 4 5 6 7 8 9 10 11 12



reverse postorder for use in second `dfs()`

1 0 2 4 5 3 11 9 12 10 6 7 8

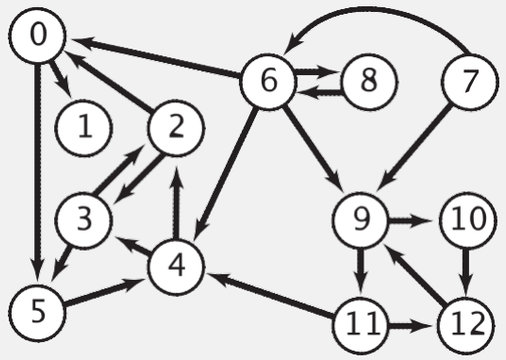
```
dfs(0)
  dfs(6)
    dfs(8)
      check 6
    8 done
    dfs(7)
    7 done
  6 done
  dfs(2)
    dfs(4)
      dfs(11)
        dfs(9)
          dfs(12)
            check 11
            dfs(10)
              check 9
            10 done
          12 done
          check 7
          check 6
  . . .
```

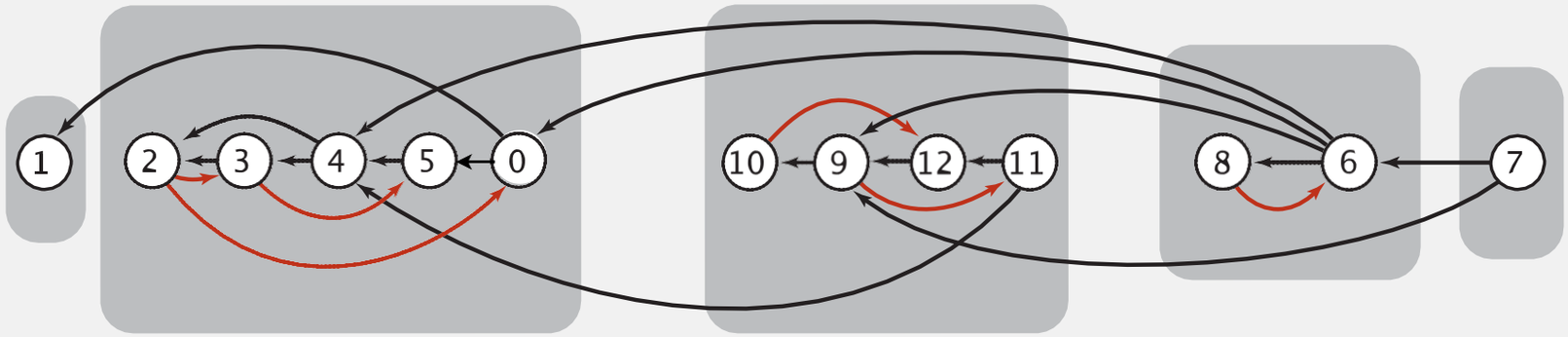
Kosaraju-Sharir algorithm

Simple (but mysterious) algorithm for computing strong components.

- Phase 1: run DFS on G^R to compute reverse postorder.
- Phase 2: run DFS on G, considering vertices in order given by first DFS.

DFS in original digraph G





check unmarked vertices in the order

1 0 2 4 5 3 11 9 12 10 6 7 8

```
dfs(1)
1 done
```

```
dfs(0)
  dfs(5)
    dfs(4)
      dfs(3)
        check 5 0
        dfs(2)
          check
          check 3
        2 done
      3 done
      check 2
    4 done
  5 done
  check 1
0 done
check 2
check 4
check 5
check 3
```

```
dfs(11)
  check 4
  dfs(12)
    dfs(9)
      check 11 12
      dfs(10)
        check
      10 done
    9 done
  12 done
11 done
check 9
check 12
check 10
```

```
dfs(6)  9
  check
  check 4
  dfs(8)
    check 6
  8 done
  check 0
6 done
```

```
dfs(7)
  check 6
  check 9
7 done
check 8
```

Kosaraju-Sharir algorithm

Proposition. Kosaraju-Sharir algorithm computes the strong components of a digraph in time proportional to $E + V$.

Pf.

- Running time: bottleneck is running DFS twice (and computing G^R).
- Correctness: tricky, see textbook (2nd printing).
- Implementation: easy!

Connected components in an undirected graph (with DFS)

```
public class CC
{
   private boolean marked[];
   private int[] id;
   private int count;

   public CC(Graph G)
   {
      marked = new boolean[G.V()];
      id = new int[G.V()];

      for (int v = 0; v < G.V(); v++)
      {
         if (!marked[v])
         {
            dfs(G, v);
            count++;
         }
      }
   }

   private void dfs(Graph G, int v)
   {
      marked[v] = true;
      id[v] = count;
      for (int w : G.adj(v))
         if (!marked[w])
            dfs(G, w);
   }

   public boolean connected(int v, int w)
   {  return id[v] == id[w];  }
}
```

Strong components in a digraph (with two DFSs)

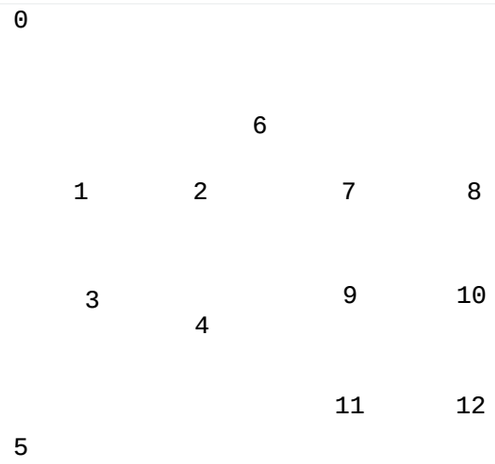
```
public class KosarajuSharirSCC
{
   private boolean marked[];
   private int[] id;
   private int count;

   public KosarajuSharirSCC(Digraph G)
   {
      marked = new boolean[G.V()];
      id = new int[G.V()];
      DepthFirstOrder dfs = new DepthFirstOrder(G.reverse());
      for (int v : dfs.reversePostorder())
      {
         if (!marked[v])
         {
            dfs(G, v);
            count++;
         }
      }
   }

   private void dfs(Digraph G, int v)
   {
      marked[v] = true;
      id[v] = count;
      for (int w : G.adj(v))
         if (!marked[w])
            dfs(G, w);
   }

   public boolean stronglyConnected(int v,  int w)
   {  return id[v] == id[w];  }
}
```

Digraph-processing summary: algorithms of the day

single-source reachability in a digraph		DFS
topological sort in a DAG		DFS
strong components in a digraph		Kosaraju-Sharir DFS (twice)